

现在假定已经写好了膨胀操作，主算法可以这样设计：首先让输入多边形往外“膨胀”，得到一个带洞多边形（如果没有洞则无解），则每个洞都是一个机器人圆心可以出现的区域。需要注意的是，这个“洞”可能退化成线段甚至是点（如题目中左图的例子）。为了避免出问题，最好是把膨胀的偏移值缩小一点。

然后计算每个区域的可清洁面积，方法是再次“膨胀”，然后计算面积。注意，两次膨胀得到的“多边形”都可能是带圆弧的，需要把直线段和圆弧都打断。

本题的算法虽然概念简单，但是实现起来还是颇有难度的，建议读者编程实践。

12.2 难题选解

12.2.1 数据结构

例题 12-11 航班 (Flights, ACM/ICPC NEERC 2012, UVa1520)

某国在一条直线上进行军事演习。有 n ($n \leq 50000$) 个导弹，用 p, x, y 3 个整数表示 ($0 \leq p \leq x \leq 50000, 0 \leq y \leq 50$)，表示起点是 $(p, 0)$ ，沿着对称的抛物线飞行，如图 12-30 所示，最高点是 (x, y) 。导弹按照输入顺序依次发射，相邻两个导弹的时间间隔是 1 分钟，而导弹飞行本身瞬间完成。

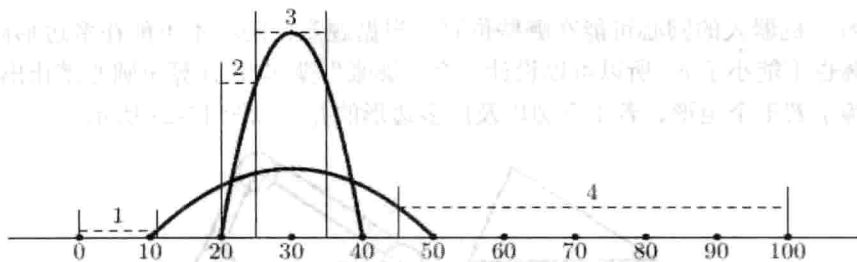


图 12-30 导弹飞行轨迹

另外还有 m 架飞机 ($1 \leq m \leq 20000$)，每架飞机用 4 个整数 t_1, t_2, x_1, x_2 表示，即飞行时间为 $t_1 \sim t_2$ ($1 \leq t_1 \leq t_2 \leq n$ ，其中第一个导弹的发射时刻为 1，最后一个导弹的发射时刻为 n)， x 坐标为 $x_1 \sim x_2$ ($0 \leq x_1 \leq x_2 \leq 50000$)。你的任务是为每架飞机计算出最小飞行高度 h ，使得时间区间 $[t_1, t_2]$ 内所有导弹轨迹在 x 坐标 $x_1 \sim x_2$ 的范围内高度都不超过 h 。如果这个范围没有导弹，则最小高度定义为 0。

【分析】

建立一棵线段树，叶结点中保存一个导弹的轨迹（抛物线），每个非叶结点 u 保存的是一个连续的导弹区间 $[m_1, m_2]$ 中所有轨迹的“轮廓线”，如图 12-31 所示。

不难看出，这是一棵关于“时间”的线段树。对于每架飞机 (t_1, t_2, x_1, x_2) ，可以按照传统的区间分解的方式，转化为对 $O(\log n)$ 条轮廓线的 $\max(x_1, x_2)$ 查询（即在 $[x_1, x_2]$ 上的最大值）。

为了让轮廓线支持 $\max(x_1, x_2)$ ，需要用一个合适的数据结构表示轮廓线。抛物线之间的

交点把轮廓线分成了若干段, 其中每个部分是一个导弹轨迹的一部分, 因此可以用五元组 (a, b, c, x_1, x_2) 表示抛物线 $y=ax^2+bx+c$ 在 $[x_1, x_2]$ 中的部分, 而轮廓线就是上述“抛物线片段”的序列 (中间可能会有空白区域)。

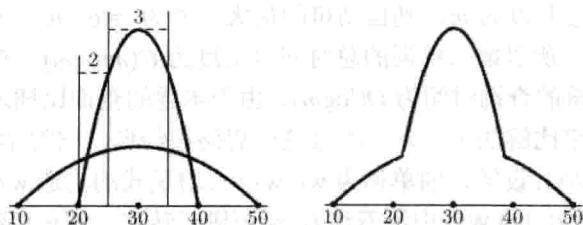


图 12-31 所有轨迹的轮廓线

只要把这些序列按照从左到右的顺序保存, 然后创建一棵线段树 (叶结点是抛物线片段), 就可以在 $O(\log n)$ 时间内求出 $\max(x_1, x_2)$ 。而一共需要查询 $O(\log n)$ 条轮廓线, 因此查询复杂度为 $O(\log^2 n)$ 。

最后考虑建树部分的时空复杂度。对于一个包含 k 个抛物线的轮廓线, 使用类似于归并排序的方法, 可以在 $O(k \log k)$ 的时间构造出一个空间为 $O(k)$ 的线段树, 因此总的时间复杂度为 $O(n \log^2 n)$, 空间复杂度为 $O(n \log n)$ 。

例题 12-12 背单词 (GRE Words Revenge, ACM/ICPC Chengdu 2013, UVa1676)

为了准备 GRE 考试, 你打算花 n ($n \leq 10^5$) 天时间背单词。每天可以做两件事之一:

□ $+w$: 学一个单词 w 。

□ $?t$: 读一篇文章 t , 统计 t 有多少个连续子串是学过的单词。

为了简单起见, 单词都是 01 串。学的单词长度总和不超过 10^5 , 文章总长度不超过 $5 \cdot 10^6$ 。

【分析】

最容易想到的算法就是维护“学过的所有单词”的 AC 自动机。由于 AC 自动机并不支持“快速插入新字符串”的操作, 所以每次学到一个新单词 w 之后, 必须重建 AC 自动机。这样, 虽然“ $?t$ ”操作非常高效 (文章 t 中的每个字符只需 $O(1)$ 时间), 但重建 AC 自动机的开销是巨大的。如果一共学了 k 个单词, 每个单词的长度均为 L , 则时间复杂度高达 $L+2L+3L+\cdots+kL=O(k^2L)$, 系统是无法承受的。幸运的是, 本题至少有 3 种高效解法, 而且都有不错的启发性。

解法 1: 维护两个 AC 自动机 `big` 和 `small`, 每次学到一个新单词后合并到 `small` 里, 等 `small` 的字符总数超过一定数值后, 合并到 `big` 里 (并且清空 `small`)。查询时把 `big` 和 `small` 分别查一遍, 加起来即可, 因此查询是每个字符 $O(1)$ 的。

假设每个单词都是单字符的, 一共有 m 个单词。当 `small` 中的字符总数超过 k 时合并, 则每 k 次操作可以看的是一轮操作。时间复杂度为:

□ 更新 `small`: $1+2+\cdots+k=O(k^2)$ 。

□ 更新 `big`, 清空 `small`: 第 i 轮为 $O(i \cdot k)$ (为了方便分析, 假设第一轮也重建了 `big`, 虽然实际上不需要)。

一共有 m/k 轮, 所以总时间复杂度为 $m/k \cdot O(k^2) + k \cdot O((m/k)^2) = O(mk + m^2/k) = m(k + m/k)$ 。当 k 和 m/k 相近时最好, 时间复杂度为 $O(m^{1.5})$ 。

解法2：用多个 AC 自动机，字符个数分别为 1, 2, 4, 8, 16, 32, 64, …，编号为 0, 1, 2, …，即编号为 i 的自动机的“理论”大小（即字符总数）为 2^i 。当自动机 i 的大小超过 2^i 时，把它所包含的字符串全部插入到自动机 $i+1$ 中，并且清空自动机 i 。

假设所有单词的总长度为 m ，则自动机的最大编号为 $t = \log_2 m$ 。每个单词最多在自动机 0, 1, …, k 里各待一次，所以插入单词的总时间复杂度为 $O(m \log m)$ 。查询时需要在每个自动机里找，所以每个字符的查询时间为 $O(\log m)$ 。由于本题的查询比插入多一个数量级，所以解法2的实际运行效率比解法1略差。不过这个思路很经典，值得学习。

解法3：使用 DAWG。设学习的单词为 $w_1, w_2, \dots$ ，增量式的构造 $w_1\$w_2\$w_3\dots$ 的 DAWG。对于“?”操作，依次在 DAWG 中沿着边 $t_1, t_2, \dots$ 进行转移。假设已经走了边 t_i ，当前状态为 S ，所要统计的是 $t[1\dots i]$ 有多少个后缀是学过的单词。根据前面的讨论，一个状态的最长单词的所有后缀就是当前状态及其在 $T(w)$ 树中所有祖先状态的字符串集。但是 $t[1\dots i]$ 不一定是 S 的最长单词，所以需要统计两项内容：

- 在状态 S 中，长度不超过 i 的所有串的权值之和（学过的单词权值为 1，其他串权值为 0）。
- 状态 S 在 $T(w)$ 中所有祖先状态的所有串的权值之和。

对于第一点，在 DAWG 的每个状态中保存一棵平衡树即可。第二点要困难一些：由于在 DAWG 的构造算法中需要动态修改 $T(w)$ 中各个结点的父指针，所以需要用一个 Link-Cut 树来维护 $T(w)$ ，从而支持“一个状态的所有祖先状态的权值之和”。其实还有一个相对容易的方法可以代替动态树：用平衡树来维护 $T(w)$ 的 DFS 序列。这里的 DFS 序列很像欧拉序列，不过记录的不是结点名称，而是带符号的权值，入栈时为正，出栈时为负。这样，DFS 序列的前缀和就是从根结点到该结点的路径上所有结点的权值之和，并且“修改父亲指针”对应着把 DFS 序列的一个子序列剪切并粘贴到另外一个位置。在《训练指南》中已经介绍如何用伸展树高效地实现这一操作。

例题 12-13 瓦里奥世界 (Rujia Liu Loves Wario Land!, Rujia Liu's Present 3, UVa11998)

很久很久以前，瓦里奥世界只有一些废弃的矿山，但没有任何连接这些矿山的道路。已知各个矿山的初始矿藏值 V_i ，你的任务是按顺序执行 m 条指令，根据要求输出所求结果。操作指令及说明如表 12-3 所示。

表 12-3 操作及含义

操 作	含 义
1 x y	修建一条直接连接 x 和 y 的道路。如果 x 和 y 已经连通（直接或者间接都算），则忽略此命令
2 x v	把矿山 x 的矿藏值改为 v （可能是因为发现了新宝物，或者一些宝物被盗）
3 x y v	统计 x 和 y 的简单路径上（包括 x 和 y 本身）有多少座矿山的矿藏值不超过 v ，然后把这些矿藏值乘起来，输出乘积除以 k 的余数。如果满足条件的矿山不存在，则输出一个 0（而不是 00 或者 01）

限制： $1 \leq n \leq 50000$ ， $1 \leq m \leq 100000$ ， $2 \leq k \leq 33333$ 。对于每条指令， $1 \leq x, y \leq n$ ， $1 \leq v \leq k$ 。输入文件大小不超过 10MB。

为了防止对所有指令进行预处理，本题的真实输入在前述输入格式基础上进行了“加

密”，即输入的各条指令中除了“类型”之外的其他值 (x 、 y 、 v) 都增加了 d ，其中 d 是在处理此指令之前上一个输出的整数（如果在此指令之前并未输出过任何指令， $d=0$ ）。

【分析】

这是一道综合性很强的题目，而且要求在线算法。维护树上信息的方法主要有欧拉序列、动态树和树链剖分 3 种，但由于操作 3 的特殊性，动态树和欧拉序列都很难起作用：如果采用动态树，需要在 $O(1)$ 时间内根据左右子树的信息计算父结点的信息。遗憾的是，操作 3 涉及的信息太复杂，通常需要树套树或者块链表实现，无法简单维护；如果采用欧拉序列，维护的信息需要满足区间减法。遗憾的是，操作 3 涉及的信息不满足。

看来只能从树链剖分入手。首先不考虑操作 1，只处理修改（操作 2）和查询（操作 3）。用块链表维护每条重路径，如图 12-32 所示。每个块里最多保存 B 个结点，按照矿藏值从小到大排序，其中 $ID[i]$ 表示价值第 i 小 ($i \geq 1$) 的结点编号， $prod[i]$ 表示价值前 i 小的结点的价值乘积。为了高效地执行链的分裂与合并（见后），不同块之间形成双向链表。

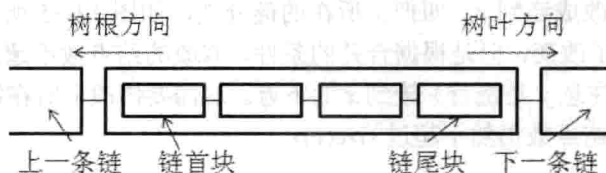


图 12-32 用块链表维护每条重路径

修改操作 ($2 \times v$)。首先要找到 v 所在的块 b ，然后重建块 b ，即把所有结点按照价值排序，重新计算前缀积和。“重建块”这个过程在其他地方也会用到，将其称为 $process(b)$ 。

查询操作 ($3 \times y \times v$)。设答案为 $res1$ 和 $res2$ ，初始时 $res1=0$ ， $res2=1$ 。首先按照 LCA 的思路，每次把 x 和 y 中靠下方的结点往上“提”，即统计 x 到 x 所在链的首结点之间的路径，更新答案 $res1$ 和 $res2$ ，然后把 x 改成 x 上一条链的尾结点，直到 x 和 y 移到同一位置，即二者的 LCA，如图 12-33 所示。

这样，问题就转化为了一系列的 $update(a, b, v, res1, res2)$ 调用，表示已知 a 和 b 在同一个链中，统计 a - b 路径上所有价值不超过 v 的结点，个数加到 $res1$ 中，乘积乘到 $res2$ 中。注意本题的权值在结点上，所有轻边是完全不用考虑的。

如图 12-34 所示， $update(a, b, v, res1, res2)$ 可以这样实现：在 a 和 b 所在的块中需要暴力查找，即枚举块内的所有结点，把所有高度在 a 和 b 之间且价值不超过 v 的结点找出来。 a 和 b 之间的块因为是完整块，所以可以二分查找，找到价值不超过 v 的结点个数 i ，则 $prod[i]$ 就是这些结点的价值乘积。

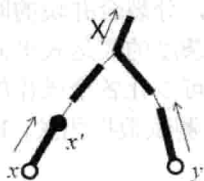


图 12-33 查询操作

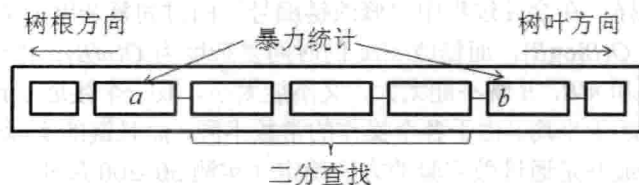


图 12-34 实现 $update(a, b, v, res1, res2)$

为了简单起见，每个结点 u 只记录链编号 $C(u)$ ，而不记录块编号，因此修改操作中需要先花 $O(L/B)$ 时间找到 u 所在的块，然后用 $O(B \log B)$ 时间重建块。查询操作最多需要调用 $O(\log n)$ 次 update 函数，而 update 函数的时间复杂度为 $O(L/B * \log(B) + B)$ 。

操作 1 的出现意味着树是会合并的，因此上面的讨论还不够。好在道路只增不减，所以可以用启发式合并，即每次把小树合并到大树中，则每个结点最多参与 $O(\log n)$ 次合并^①。这样，问题的关键就在于如何高效地合并两棵树的树链剖分。

执行操作 1 $x\ y$ 时，首先找到 x 和 y 所在树的树根，如果相同，则忽略本操作；否则假设 x 所在的树结点比较多， y 所在的树的结点比较少（否则可以交换 x 和 y ）。接下来，需要把 y “嫁接”到结点 x 处。但是由于 y 所在树的树根可能是其他结点，首先要把 y 所在的树以 y 为树根重建（包括重建树链剖分），然后设 x 为 y 的父结点。

接下来是重头戏了：由于 x 多了一棵子树 y ，所以 x 往下的重边有可能会变化。例如， x 是叶子，或者 x 原来的重边子结点 $W(x)$ 的子树没有 y 的子树大，即 $\text{size}(W(x)) < \text{size}(y)$ 。那么 x 往下的重边需要改成连到 y ，即把 x 所在的链分裂，如图 12-35 所示。 L 部分所有结点的“链编号”都发生了改变，但是根据合并的条件，修改的结点数不超过 $\text{size}(y)$ 。分裂之后还要把 y 所在的链（注意 y 是链首）接到 x 的下方。这需要修改 y 所在链的所有结点的“链编号”，但是修改的结点数仍然不超过 $\text{size}(y)$ 。

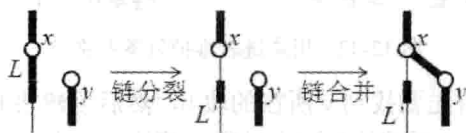


图 12-35 将 x 所在的链分裂

最后是修改 x 及其所有祖先 p 的 $\text{size}(p)$ 。 x 的祖先可能很多，不能一一修改，而只能一个块一个块地修改，即每个块设一个懒标记，表示该块所有结点的整体 size 增量，当访问 size 时再删除标记。这里有一个关键问题： x 的所有祖先的 size 都变大了，所以它们到父结点的边可能会从轻边改成重边，因此还需要一些复杂的操作。幸运的是，此处并不需要严格地使用树链剖分的定义，而是可以让这些轻边保持原样。因为每个结点到根的路径上仍然最多有 $O(\log n)$ 条链，所以时间复杂度并不会变坏。这样，通过分裂链、合并链和修改 size 这 3 个步骤即完成了两棵树的合并。

还有两个细节没有提到：分裂链时需要分裂 x 所在的块，而在合并链时需要试着合并 x 和 y 所在的两个块（它们是相邻块）。根据块链表的一般思路，只有当这两个块在合并之后仍然不超过 B 时才合并。

这样，在合并过程中“修改链编号”的时间复杂度为 $O(\text{size}(y))$ ，分裂合并块的时间复杂度为 $O(B \log B)$ ，而修改 size 的时间复杂度为 $O(n/B)$ 。由于时间复杂度的表达式里同时出现了 B 和 n/B ， B 既不能太大，又不能太小，取一个接近 $\sqrt{B}$ 的值可以让各个操作的时间复杂度趋于平均。由于各个操作的常数不同，而且链的实际长度还和测试数据相关， B 的最佳取值最好是通过做实验的方法确定（实测 50~300 最佳）。

^① 《训练指南》中的“图询问”问题也用到了这个技巧。

12.2.2 网络流

例题 12-14 芯片难题 (Chips Challenge, ACM/ICPC World Finals 2011, UVa1104)

作为芯片设计的一部分，你需要在一个 $N \times N$ ($N \leq 40$) 网格中放置部件。其中有些格子已经放了部件（用 C 表示），还有些格子不能放部件（用 “/” 表示），剩下的格子需要放置尽量多的新部件（用 W 表示）。

要求对于所有 $1 \leq x \leq N$ ，第 x 行的部件个数（C 和 W 之和）等于第 x 列的部件个数。为了保证散热，任意行或列的部件个数不能超过整个芯片总部件数的 A/B 。如图 12-36 所示，若 $A/B=3/10$ ，则图 12-36 (a) 的最优解如图 12-36 (b) 所示，一共放置了 7 个新部件。

CC/..	CC/W.
././.	W/W/./
..C.C	W.C.C
/C..	/CWW
/./C/	/W/C/
(a)	(b)

图 12-36 放置部件的最优解

【分析】

根据经验，构造一个二分图，左边是行，右边是列，一个部件就是一条边 $X_i \rightarrow Y_j$ 。如何表示第 i 行的总流量等于第 i 列呢？从 Y_i 再连一条边到 X_i 即可。因为每个 Y 结点的出弧只有一条（到 X_i ），而每个 X_i 只有一条入弧（从 Y_i ），所以 X_i 的流量肯定等于 Y_i 的流量。进一步分析可发现：其实这样做等价于把 X_i 和 Y_i “粘” 起来。也就是说，根本不需要构造二分图，一共 n 个结点即可。一个部件 (i, j) 就是有向弧 $i \rightarrow j$ 。如果在 (i, j) 上加上一个费用 1，则总费用就是新部件的个数。这样就转化为了求最大费用循环流问题，用第 11 章中介绍的方法求解即可。

接下来还需要加上题目中的两个限制。首先是必须有流量的边，也就是 C 对应的边。有两种做法，一是设容量下界也是 1，二是设 cost 为负无穷。接下来考虑每行每列 A/B 的限制。方法是枚举每行/列部件数的最大值 m ，给每个点增加结点容量 m （然后用标准方法拆成两个点），然后求最大费用循环流，看看费用是否至少为 $m \cdot B/A$ 。注意， m 的值只有 $0 \sim n$ 这 $n+1$ 种可能，所以时间复杂度只需乘以 $O(n)$ ，仍然可以承受^①。

例题 12-15 《第七夜》、《时空轮回》与水的故事 (Never7, Ever17 and Wat[j]er, Rujia Liu's Present 6, UVa12567)

有一个 n 个点、 m 条有向边的网络，每条边都有容量上下界 b 和 c ，求一个循环流，使得所有边中的最大流量和最小流量之差尽量小。 $n \leq 50$, $m \leq 200$ 。

【分析】

本题虽然是网络流问题，但是“最大流量和最小流量之差”似乎无法对应到经典的网络流模型中。怎么办呢？

很多图论优化问题，包括最短路、最大流和最小费用流等，都可以用线性规划建模，本题是不是也可以呢？下面尝试一下。设第 i 条边的流量为 x_i ，则容量限制可以列出两个不等式，对于每个结点可以列出流量平衡“等式”，目标是最小化 $\max\{x_i\} - \min\{x_i\}$ 。问题还是出现在同一个位置：目标函数不是变量的线性组合，不符合“线性规划”的定义。

既然线性规划模型比较灵活，现在我们对目标函数进行代数变形。再引入两个变量

^① 仔细分析后可以发现：因为流量可以复用，所以其实复杂度连 $O(n)$ 都不需要乘。不过对于本题的规模，这个优化不是必需的。

$A = \min\{x_i\}$, $B = \max\{x_i\}$, 然后对每个 x_i 添加不等式 $A \leq x_i \leq B$, 则目标变成了最小化 $B - A$, 符合线性规划模型。可是这能不能保证算出来最优解真的满足 $A = \min\{x_i\}$, $B = \max\{x_i\}$ 呢? 如果不满足, 例如, $A < \min\{x_i\}$ (根据不等式约束, $A \leq \min\{x_i\}$), 那么把 A 改成 $\min\{x_i\}$ 之后, 约束仍然满足, 并且目标函数变得更小, 与最优解矛盾。因此, 变形后的线性规划模型可以得到原题的最优解。

例题 12-16 怪兽滴水嘴 (Gargoyle, ACM/ICPC Xi'an 2006, UVa12110)

城堡顶层有 n 个怪兽状滴水嘴, 还有一个包含 m 个连接点和 k 个水管的水流系统 ($1 \leq n \leq 25$, $1 \leq m \leq 50$, $1 \leq k \leq 1000$)。从滴水嘴流出的水直接进入蓄水池, 通过水管后重新由滴水嘴流出。假设水量无损失, 每个连接点处的总入水速度应该等于总出水速度。水管中水流的速度有上下界, 单位水速有固定费用。

你的任务是设计各水管的水速, 用尽量少的总费用让各滴水嘴的出水速度相同。

每个水管用 5 个整数 a, b, l, u, c 表示 ($0 \leq a, b \leq n+m$, $0 \leq l \leq u \leq 100$, $1 \leq c \leq 100$), 即每个水管入口和出口编号 (蓄水池编号为 0, 滴水嘴编号为 $1 \sim n$, 连接点编号为 $n+1 \sim n+m$), 水速下限、上限, 以及单位水速的费用。水管不会连接两个相同点, 即水管入口不会是滴水嘴, 出口不会是蓄水池。每两个点之间最多一条水管 (如果有水管从 a 到 b , 则不会再有其他水管也从 a 到 b , 也不会有水管从 b 到 a)。输入结束标志为一个 0。

【分析】

根据题意, 蓄水池的编号为 0。把它拆成两个点 0 和 0', 则本题的模型就是求一个最小费用流, 使得进入 0' 点的所有流量均相同。根据题目背景, 把那些流入蓄水池的弧称为“瀑布弧”。下面来看一个例子。

如图 12-37 所示, 除了弧 $0 \rightarrow 4$ 的容量上下界均为 1 之外, 其他弧的容量下界为 0, 上界为无穷大。所有水管的单位费用为 1 (注意, 瀑布弧的费用为 0)。不难发现这个例子的唯一可行解如图 12-38 所示 (边上的数代表流量)。

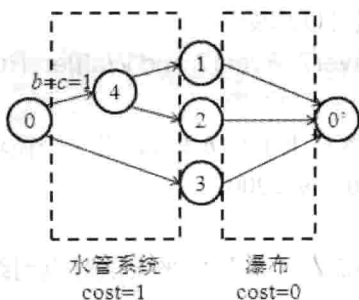


图 12-37 瀑布弧

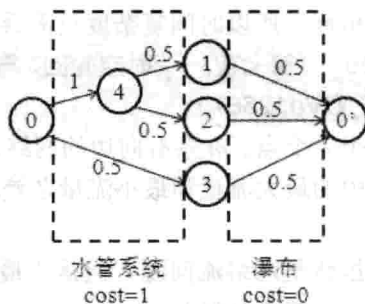


图 12-38 唯一可行解

从图 12-38 可知, 出现了非整数的流量。这样一来, 就无法在修改模型之后只求一次费用流就得到最终结果, 只能寄希望于参数搜索——先确定瀑布弧的相同流量 f , 然后再求出对应的最小费用 $c(f)$ 。这样的想法是可行的, 因为 f 确定下来以后问题就会转化为普通的带上下界最小费用流问题。这样, 就需要把注意力集中在函数 $c(f)$ 上。

首先考虑 f 的可行域。不难证明 f 的可行域为连续区间 $[\text{left}, \text{right}]$, 因此可以用二分法确

定这个可行域的边界：给瀑布弧设置下界 0 和上界 f ，如果网络没有可行流，则说明 $f < \text{left}$ ；如果网络有可行流但有的瀑布弧不满载，则说明 $f > \text{right}$ （想一想，为什么）。

接下来怎么办？直接输出最小流对应的费用？很可惜，最小的 f 并不对应最小的费用。下面的例子很好地说明了这一点。

有两条弧的上下界均为 1，因此流量必须为 1。如果要 f 最小，应该沿着 $0 \rightarrow 2 \rightarrow 3 \rightarrow 1 \rightarrow 0'$ 的顺序流动，但这样一来，经过了费用 100 的弧。另一方面，如果沿着 $0 \rightarrow 2 \rightarrow 1 \rightarrow 0'$ 和 $0 \rightarrow 3 \rightarrow 1 \rightarrow 0'$ 流动，虽然流量 2 不是最小的，但费用仅为 4，如图 12-39 所示。

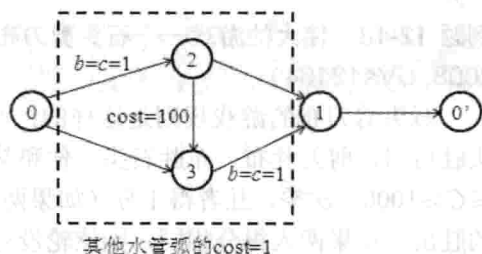


图 12-39 最小的 f 不对应最小的费用

《训练指南》中介绍过“流量不固定的最小费用流”问题，并且指出费用是流量的下凸函数。这个结论在本题中也成立，即在可行域内 $c(f)$ 是 f 的下凸函数，因此用三分法求解即可^①。

本题是笔者为 2006 年 ACM/ICPC 西安赛区所命的题目，上述算法便是笔者当时给出的“标准算法”。虽然概念并不复杂，但是毕竟包含二分、三分以及容量有下界的最小费用流问题等诸多因素，用程序实现并不容易。看到这里，聪明的你是否能想到一个“取巧”的方法呢？没错，可以用线性规划方法！只需要加一些“瀑布弧流量全相等”的等式，本题就转化成了线性规划问题。不过，这个新算法和刚才介绍的传统方法相比，效率如何呢？读者不妨一试。

12.2.3 数学

例题 12-17 简单加密法 (Simple Encryption, ACM/ICPC Kuala Lumpur 2010, UVa12253)

输入 K_1 ($0 < K_1 < 50000$)，解方程 $K_1^{K_2} \equiv K_2 \pmod{10^{12}}$ ，即 K_1 的 K_2 次方的十进制末 12 位等于 K_2 。注意， K_2 的十进制必须恰好包含 12 个数字，不能有前导 0。输入保证有解。

【分析】

很多数学题除了需要知识和技巧之外，还需要经验和直觉（而计算机是验证“直觉”的绝好工具！），本题便是一例。本题的模 10^{12} 很大，不妨先缩小一点，例如，把模改成 10^3 ，那么 K_2 的取值范围是 100~999，直接枚举即可。取 $K_1=123$ ，不难枚举到唯一解是 547。如果把模改成 10^4 ，可以枚举到唯一解是 2547。会不会是巧合？再换一个 $K_1=234$ ，可以枚举到模为 10^3 时的唯一解是 616， 10^4 时的唯一解为 1616。还有更神奇的：123⁵⁴⁷ 的末 4 位为 2547，而 234⁶¹⁶ 的末 4 位是 1616！

看上去可以得到一个猜想：如果 K_1^n 以 dn 结尾，则 K_1^{dn} 也以 dn 结尾。这里 dn 是指把数字 d 放在 n 前面的数。试着验证一下：123²⁵⁴⁷ 的末 5 位是 92547，123⁹²⁵⁴⁷ 的末 6 位是 692547。123⁶⁹²⁵⁴⁷ 的末 7 位是 1692547。看上去很不错。如果这个结论是对的，那么只需要用暴力法求出一个很小的 n 使得 K_1^n 以 n 结尾，然后用这个结论不断地往 n 的前面加数字，直到它拥

^① 本题还有一个有意思的结论：最小费用对应的 f 一定是有理数，且分母不超过 n （即滴水嘴的数量）。这个结论并不容易证明，有兴趣的读者可以一试。

有 12 个数字为止——然后祈祷最后加上的那个数字不是 0。这就是最终算法。

用数学归纳法可以证明上述结论^①，不过比赛当中通常无暇考虑。只要最终算法够简单，写程序的时间很可能还没有证明的时间长。即使写出来的程序是错的，也没有耽误太多的时间。

例题 12-18 伟大的游戏——石头剪刀布 (The Great Game, ACM/ICPC Kuala Lumpur 2008, UVa12164)

石头剪刀布的游戏规则是这样的：两个人一起出拳，必须出石头、剪刀、布之一。石头胜剪刀，剪刀胜布，布胜石头。你和某人玩石头剪刀布游戏，分为若干轮，每轮出 G ($1 \leq G \leq 1000$) 次拳。胜者得 1 分（如果两个人出的一样，都不得分）。每轮结束后，得分多的胜出（如果两人得分相同，则该轮没有人胜出）。当你的对手比你多赢 L 轮时，你就算输掉了整个比赛；当你比对手多赢 W 轮时，你就算赢得了整个比赛。你的任务是找一个最优策略，使赢得整个比赛的概率最大。 $1 \leq W, L \leq 100$ 。

假定你的对手的策略是固定的，而且每轮都一样：第 i 次出拳时分别有 $a_i\%$, $b_i\%$, $c_i\%$ 的概率出石头、布和剪刀。输入保证 $a_i + b_i + c_i = 100$ 。

【分析】

你的任务是比对手多赢 W 轮，而各轮之间是不相关的，所以你需要每一轮都玩得尽量好。可是什么叫“玩得尽量好”呢？如果每一轮只有赢和输两种可能，那么“玩得尽量好”就是指获胜的概率尽量大。但是在本题中，每一轮除了输赢之外还有可能是平局。如果有两种策略，一种是 20% 概率赢，80% 概率平（因此不可能输），但另外一种 80% 概率赢，10% 概率平（因此还有 10% 的概率输），哪种策略更好呢？仔细思考后会发现：虽然第一种策略的胜率比较低，但它是必胜的（即答案是 100%）——对手没有任何机会获胜；第二种策略虽然赢的概率比较大，但却有概率输掉，如果 $L=1$ ，答案肯定不是 100%。

《训练指南》中曾经介绍过马尔科夫链。如果用一个编号为 x 的结点表示“比对手多赢 x 场”这个状态，则本题就是一个包含 $L+W+1$ 个结点（即 $-L, -(L-1), \dots, 0, 1, \dots, W$ ）的马尔科夫链，要求一个策略使得结点 0 首达结点 W 的概率最大。

假设最优策略使得每局获胜的概率为 p_{win} ，输掉的概率为 p_{lose} ，每个内结点（即不是 $-L$ 也不是 W 的结点）往左的转移概率为 p_{lose} ，往右转移的概率为 p_{win} ，转移到自己的概率为 $(1 - p_{\text{win}} - p_{\text{lose}})$ 。因为本题并不关心到达结点 $-L$ 或 W 的具体时间，只关心先到达 W 的概率，所以刚才的马尔科夫链等价于去掉自环（即每个状态到自身的转移），然后把往左往右的概率归一化（即让二者加起来等于 1）。此处要最大化的正是这条新马尔科夫链中的获胜概率，即 $p_0 = p_{\text{win}} / (p_{\text{win}} + p_{\text{lose}})$ 。

至此，问题分成了两个完全独立的部分：如何最大化 p_0 ，以及已知 p_0 之后如何求出状态 W 的首达概率。后者的一般做法如下：设状态 i 时的获胜概率为 $d(i)$ ，根据边界 $d(-L)=0$ ， $d(W)=1$ 以及马尔科夫方程联立求解。具体解法在《训练指南》中已有详细叙述。对于本题中特殊的马尔科夫链，还可以直接求出解的封闭形式。另外，还可以用迭代法而非高斯消元法求解方程组，这里不再详述。

^① 事实上，还可以证明一个更强的结论：如果不考虑“ K_2 不能有前导 0”这个条件， K_2 是唯一存在的。

前者也有两种解法：二分法和不动点迭代法。不动点迭代法及其收敛性的证明超出了本书的讨论范围，因此这里只介绍二分法。二分答案 p ，看看是否有一种策略使得 $p_{\text{win}}/(p_{\text{win}}+p_{\text{lose}}) \geq p$ ，即 $(1-p)*p_{\text{win}}-p*p_{\text{lose}} \geq 0$ 。接下来就只需动态规划计算 $(1-p)*p_{\text{win}}-p*p_{\text{lose}}$ 的最大值了。令“胜”的权值为 $1-p$ ，“负”的权值为 $-p$ ，则问题转化为最大化权值的数学期望。设状态 $d(i,j)$ 表示前 i 次猜拳，得分为 j （注意 j 可能为负数）时的最大期望，分剪刀、石头、布 3 种情况讨论即可。

例题 12-19 自行车 (Cycling, ACM/ICPC NWERC 2012, UVa1677)

你有一个很棒的自行车：没有最大速度，加速度不超过 0.5m/s^2 ，但可以瞬间把速度减为 0 到当前速度之间的任意速度。 $T=0$ 时刻，你在 $X=0$ 的位置。目标位置是 $X=X_{\text{dest}}$ ($1 \leq X_{\text{dest}} \leq 10000$)。一共有 L ($0 \leq L \leq 10$) 个红绿灯，每个红绿灯用 3 个整数描述：位置 X_i ($0 < X_i < X_{\text{dest}}$)，红灯长度 R_i ($10 \leq R_i \leq 500$)，绿灯长度 ($10 \leq G_i \leq 500$)。 $T=0$ 时，所有灯刚刚变红。不同红绿灯的位置保证不同。求到达目标位置的最短时间。

【分析】

用 t - x 图（横坐标为时间，纵坐标为位置）可以直观地表示出一个合法解，如图 12-40 所示。

从图中可以得到两个直观的结论。

结论 1：通过一个点 (t,x) 时，速度越大越好，因为可以任意减速。

结论 2：不要在中间（没有红绿灯的地方）变速，且不等等待时加速度保持最大。

证明：首先考虑没有红绿灯的情况。如何保证通过点 (t,x) 时速度最大？画一个速度-时间图就一目了然了。相同时间（横轴）走相同路程（面积），而开始低速后加速最终得到的速度更高（纵轴），如图 12-41 所示。也就是说：要么就等着，要加速就要是最大加速度，并且等待/刹车一定是起点或者刚过红绿灯之后。

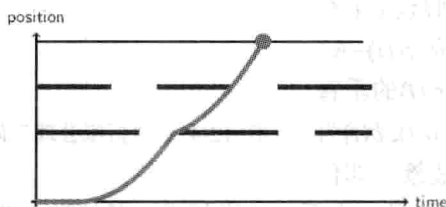


图 12-40 t - x 图

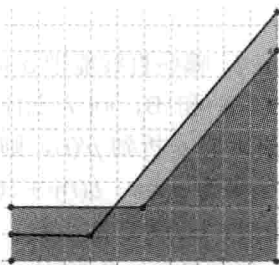


图 12-41 速度-时间图

这样就证明了，只需考虑红绿灯刚刚变化时的状态 (t,x) 。注意， x 只有 $L+1$ 种取法（起点或者某个红绿灯处），而 t 只能取该红绿灯刚刚变色的时刻（ $x=0$ 时 t 必须等于 0）。稍后将会分析状态 (t,x) 的个数，不过现在先设计算法。

设 $d(t,x)$ 表示自行车处于状态 (t,x) 下的最大速度，则可以写一个“刷表法动态规划”：枚举 (t,x) 的“下一个状态” (t',x') （其中 $t' > t$, $x' > x$ ），更新 $d(t',x')$ 。需要分两种情况讨论。

情况 1：减速但不等待。这要求解减速后的速度 v ，使得保持最大加速度行驶后恰好到达状态 (t',x') 。注意：因为行驶距离 $x'-x$ 和时间 $t'-t$ 都已经固定，且加速度恒定为 0.5 ，可

以直接解出 v 。如果 $v > d(t, x)$ ，说明这个解不合法（因为自行车不能瞬间加速！），而如果 $v < 0$ ，其实已经变成了情况 2。

情况 2：把速度减为 0，等待一段时间后重新开始加速。因为初速度为 0，加速度恒定为 0.5，根据行驶距离可以直接算出加速时间，也就能算出等待时间了。

需要特别注意的是，不管是情况 1 还是情况 2，算出具体路线以后却要判断这条路线会不会“闯红灯”。只有不闯红灯时才能用到达 (t', x') 时的速度更新 $d(t', x')$ 。另外，每个状态 $d(t, x)$ 都有可能直接最大加速冲到终点，从而更新最终答案，但也要判断有没有闯红灯。

状态有多少个呢？最坏的情况就是 10 个红绿灯把 10000 米分成 11 段，每段 910 米，且每次都要从头加速，因此行驶时间为 $11 * \sqrt{4 * 910} = 664$ 秒。另外，每个红灯处最多等 500 秒，因此总时间不超过 5664 秒，每个红绿灯最多经过 $5664 / (10 + 10) < 300$ 个周期。粗略计算一下，上述算法的计算量是可以承受的，而且刚才的估算非常“悲观”，实际上很难达到^①。

命题组最初设计的题目还要更难一点：自行车的速度还有一个上限值。有兴趣的读者可以思考一下，如何求解这个“加强版”的题目。另外，上述算法还有很大的优化余地（例如，计算 $d(t, x)$ 时不一定要枚举所有满足 $t' < t, x' < x$ 的状态 (t', x') ），有兴趣的读者可以深入思考。

例题 12-20 折纸公理 6 (Huzita Axiom 6, ACM/ICPC NEERC 2011, UVa1678)

输入两条线 l_1, l_2 和两个点 p_1, p_2 ，找一条直线 l ，使得 p_1 的对称点落在 l_1 上，且 p_2 的对称点落在 l_2 上。换句话说，如果以 l 为折纸痕， p_1 会折到 l_1 上， p_2 会折到 l_2 上，如图 12-42 所示。

输入保证 l_1, l_2 不同，但 p_1, p_2 可以相同。 p_1 不在 l_1 上， p_2 不在 l_2 上。坐标都不超过 10。如多解，输出任意解；如无解输出 4 个 0。

【分析】

给定 p, l ，哪些直线能把 p 折到 l 上呢？假设 l 上有两个不同点 A 和 B ，则 l 上任意点可以写成 $p'(t) = A + t(B - A)$ 。如果把 p 折到 $p'(t)$ ，则折纸痕为 $p - p'(t)$ 的垂直平分线，化简为 $a(t)x + b(t)y + c(t) = 0$ ，其中 $a(t), b(t)$ 为 t 的线性函数， $c(t)$ 为二次函数。这是一个直线族，即任取一个 t ，都能得到一条直线，把 p 折到 l 上。另一方面，对于任意一条能把 p 折到 l 上的直线，都存在这样一个参数 t 。此处把这个直线族记为 $(a(t), b(t), c(t))$ 。

在本题中，有两对点和两条直线，因此可以得到两个直线族 $(a_1(t), b_1(t), c_1(t))$ 和 $(a_2(t), b_2(t), c_2(t))$ 。目标是求出一条直线同时属于两个直线族，这等价于求出两个参数 t_1 和 t_2 ，使得直线 $a_1(t_1)x + b_1(t_1)y + c_1(t_1) = 0$ 和 $a_2(t_2)x + b_2(t_2)y + c_2(t_2) = 0$ 是同一条直线。

一条直线有多种表示法（例如， $x + y + 1 = 0$ 和 $2x + 2y + 2 = 0$ 是同一条直线），不能简单地认为 $a_1(t_1) = a_2(t_2)$ ， $b_1(t_1) = b_2(t_2)$ ， $c_1(t_1) = c_2(t_2)$ ，而只能认为三者“成比例”（但是要注意 0 不能做分母）。一种常见方法是将“二直线相等”变成以下两个条件：

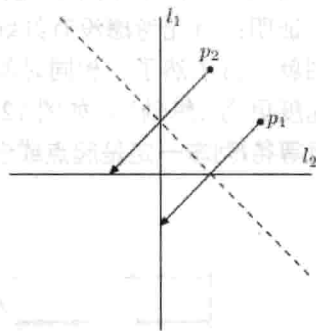


图 12-42 “折纸公理”问题示意图

^① 官方数据中的最大答案为 1685.830。

□ 法线共线, 即 $(a_1(t_1), b_1(t_1))$ 和 $(a_2(t_2), b_2(t_2))$ 共线。

□ 其中一条直线上有一个点在第二条直线上。

根据这两个条件, 可以列出两个关于 t_1 和 t_2 的方程, 消去 t_2 后, 能得到一个关于 t_1 的三次方程, 用二分法求解即可 (要注意退化情况)。

例题 12-21 简单几何 (Easy Geometry, ACM/ICPC NEERC 2013, UVa1679)

输入一个凸 n ($3 \leq n \leq 100000$) 边形, 在内部找一个面积最大, 边平行于坐标轴的矩形, 如图 12-43 所示。

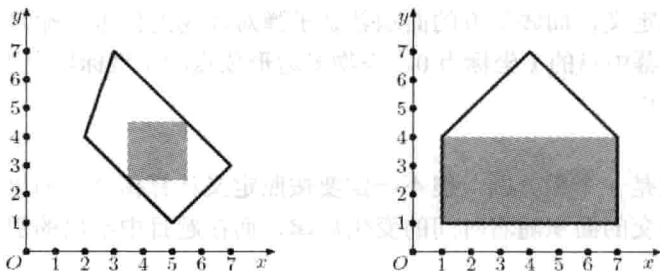


图 12-43 “简单几何”问题示意图

【分析】

虽然本题是几何题 (而且题目名称里也有“几何”字样), 但用纯几何的方法解题很难奏效。因为图形是凸的, 可以从函数的角度考虑问题。对于任意横坐标 x_0 , 竖直线 $x=x_0$ 最多和凸多边形相交于两个点, 设 $y_1(x_0)$ 和 $y_2(x_0)$ 分别为低点和高点的坐标。对于任意给定的 x_0 , 可以用二分查找的方法求出 $y_1(x_0)$ 和 $y_2(x_0)$ 。下面假设矩形的左端点为 x , 宽度为 w , 则最大矩形包含在如图 12-44 所示的阴影部分梯形中。

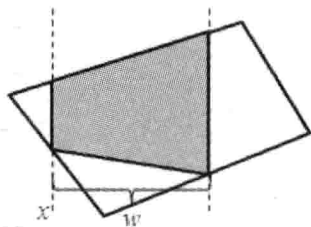


图 12-44 二分查找求出 $y_1(x_0)$ 和 $y_2(x_0)$

根据图 12-44, 最大矩形的面积 $S_1(x, w) = w * (\min\{y_2(x), y_2(x+w)\} - \max\{y_1(x), y_1(x+w)\})$ 。当 w 固定时, 上述表达式是 x 的凸函数, 所以宽度为 w 的最大矩形面积 $S_2(w)$ 可以通过三分法求出。类似地, $S_2(w)$ 也是关于 w 的凸函数, 所以最大矩形的面积也可以通过三分法求出。

12.2.4 几何

例题 12-22 打怪物 (Shooting the Monster, ACM/ICPC Kuala Lumpur 2008, UVa12162)

你正在玩一个打怪物的游戏, 其中怪物是一个巨大的不能动弹的 n ($n \leq 50$) 边形, 位于右半屏幕。你发的子弹也是一个多边形, 从左半屏幕开始匀速水平向右飞到无穷远处, 速度为 1。注意, 怪物在被子弹打穿的过程中不会产生形变, 也不会移动。

为了增加游戏的真实性, 一发子弹对怪物的伤害等于子弹与怪物的公共部分面积对时间的积分。例如, 在图 12-45 中, t 分别为 0 和 3, 相交部分的面积分别为 0 和 1。

对于上面的场景, 可以画出相交面积随时间变化的曲线, 如图 12-46 所示。



图 12-45 t 为 0 和 3 时相交部分面积

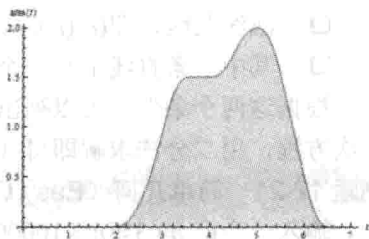


图 12-46 相交面积随时间变化的曲线

根据定积分的定义，曲线下方的面积就是子弹对怪物的伤害。输入坐标均为绝对值小于 500 的整数。屏幕中点的 x 坐标为 0，怪物多边形顶点的 x 坐标均大于 0，子弹多边形顶点的 x 坐标均小于 0。

【分析】

本题在定义上是一个积分题，但不一定要按照定义计算积分。如果按照定义，则需要分析两个多边形相交的面积随着时间的变化规律，而在题目中给出的那个曲线看上去毫无规律。怎么办呢？

因为子弹是水平向右飞行的，可以把两个多边形划分成水平条而非竖直线，则不同水平条之间的结果完全独立，依次求解后累加即可。具体来说，从两个多边形的所有顶点出发画一条水平线，则每个水平条内都是一些梯形（或退化成三角形），如图 12-47 所示。

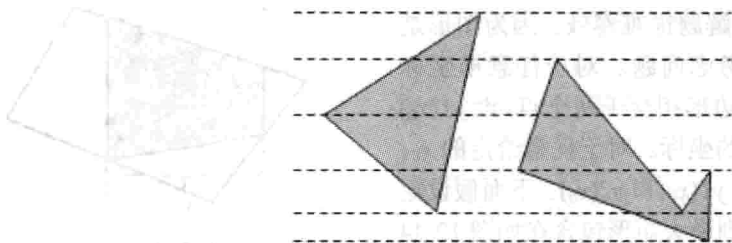


图 12-47 水平线划分出的梯形或三角形

对于一个水平条来说，同一个多边形划分出的梯形/三角形可以合并到一起（想一想，为什么），如图 12-48 所示。所以问题转化为子弹和怪物都是单个梯形的情况，可以直接求解（需要手工计算一个简单积分）。



图 12-48 子弹和怪物形状转化为梯形

例题 12-23 快乐的轮子 (Merrily, We Roll Along!, World Finals 2002, UVa1017)

你有一个圆形的轮子，放在一条由水平线段和竖直线段组成的折线道路上，轮子的中心在道路起点的正上方。在保持和折线接触的前提下，你沿着道路把轮子滚到尽头（即让轮子的中心在道路终点的正上方）。你的任务是计算圆心移动的总距离。

在下面的例子中，假定轮子半径为 2，道路第一段和最后一段的高度相同，长度都是 2。中间的水平线段长度为 2.828427，比另两条水平线段低 2 个单位。滚动轮子时，轮子首先

从位置1（起点）水平移动到位置2，然后旋转 45° 到位置3，再旋转 45° 到位置4，最后水平移动到位置5（终点），圆心移动距离为7.1416，如图12-49所示。

下面的例子更为复杂：两边是两条长度为3的水平线段，中间是一条长度为7，高度比两边低7个单位的水平线段。轮子的半径为1，移动总距离为26.142，如图12-50所示。

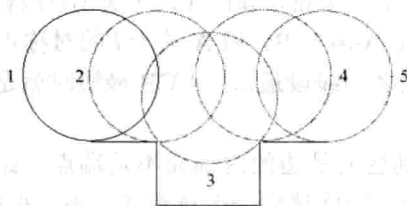


图 12-49 轮子滚动状态

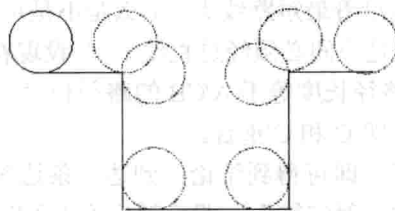


图 12-50 更复杂的轮子滚动状态

输入轮子的半径 r 和道路的段数 n ($1 \leq n \leq 50$)，以及每段道路的长度和道路右端处的高度变化值（正数代表变高，负数代表变低，最后一段道路右端的高度变化值保证为0），输出圆心移动距离，保留3位小数。输入保证第一段和最后一段道路的长度严格大于 r ，且在滚动过程中轮子不会同时碰到两条垂直道路。

【分析】

本题有两个常用算法。第一种方法类似于“清洁机器人”问题，先将道路外扩距离 R ，打散线段和圆弧，然后判断每条小线段和圆弧的中点与输入道路的距离是否小于 R ，如果是，则不要统计这条线段/圆弧，如图12-51所示。

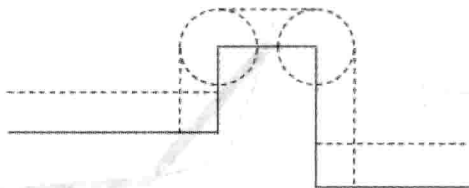


图 12-51 判断是否统计线段圆弧

这个算法比较易于理解和编写，查错也很方便，但运行速度较慢。还有一个概念上较为简单、速度快，但容易出错的算法：直接模拟。任何时刻有4个可能的状态：水平向右移动（0）、竖直向下移动（1）、竖直向上移动（2）、绕顶点顺时针旋转（3），可能的转移如图12-52所示。



图 12-52 4种可能的状态

例题 12-24 客房服务 (Room Services, ACM/ICPC World Finals 2012, UVa1286)

给定一个凸 n ($3 \leq n \leq 100$) 边形和多边形内的一个点，要求从这个点出发，到达每条

边恰好一次，然后回到起点，使得总路程尽量短。注意：到达一个点相当于到达了它所在的两条边。

【分析】

本题看上去相当困难，因为可行的路径有无穷多条。怎么办呢？物理老师曾经说过：光线总是沿着最短路线走。那么是不是可以借鉴一个光路呢？如图 12-53 所示，假设要从 A 到 B，但是中间必须经过直线 l 。假设现在的路径是 $A \rightarrow C \rightarrow B$ 。做 A 关于 l 的对称点 A' ，则 ACB 的路径长度等于 $A'CB$ 的路径长度。因为两点之间线段最短， $A'CB$ 最短时就是这三点共线时，即 C 和 C' 重合。

这样，即可得到结论：到达一条边时，只要到达的是边的内部而不是端点，路线都满足“光的反射定律”，即反射角等于入射角。另外，还能猜到一个直观（但不是很好证明）的结论：存在一个最优解，使得所有边按照逆时针顺序到达。有了这两个结论，就可以设计出主算法了。

首先枚举第一次到达的边，把环打断成线。为了方便，把第一次到达的边的终点编号为 1，其他点按照逆时针顺序依次编号为 $2 \sim n$ ，起点编号为 0，终点编号为 $n+1$ （起点和终点重合）。接下来进行动态规划：设 $d(i)$ 为表示当前点编号是 i ，还需要多长路径才能走到终点。枚举下次走到的顶点编号 j ，则：

$$d(i) = \min\{w(i,j) + d(j) \mid j=i+1 \cdots n+1\}$$

其中， $w(i,j)$ 表示从顶点 i 出发，到达顶点 j ，中途按顺序经过 $i \sim j$ 之间所有边的最短路径长度，如图 12-54 所示。

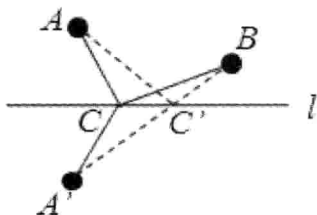


图 12-53 ACB 的路径长度最短



图 12-54 经过 $i \sim j$ 的所有边的最短路径长度

计算 $w(i,j)$ 时需要不断地计算 i 关于各条边的对称点，最后和 j 相连，然后恢复出整条折线。但是需要判断是否每次“到达一条边”时接触点都真的在线段的内部。如果接触点在线段外面，则说明这条路线是非法的， $w(i,j)$ 应设为正无穷。细心的读者可能会问：如果有接触点在线段外面，可以退而求其次，不走镜面反射路线，但也不该是正无穷啊？但其实这样做的结果是直接走到多边形的一个顶点，已经被上述动态规划算法考虑到了。

当 i 或者 j 为 0 或者 $n+1$ 时，需要一些特殊处理。另外，还要注意 $j=i+1$ 的情况。细节留给自行读者思考。

例题 12-25 最短飞行路径 (Shortest Flight Path, ACM/ICPC World Finals 2012, UVa1288)

如图 12-55 所示，地球表面有 n 个机场，要求从机场 s 飞到机场 t 时，飞行总距离最小（无解输出 impossible），且飞行过程中始终满足：离最近机场的距离不超过 R 。由于油箱限制，最大连续飞行距离为 c ，所以可能需要中途在其他机场加油。本题距离都是指球面距

离（假定飞机沿着地球表面飞行）。地球是半径为 6370km 的球，有多组询问 (s, t, c) 。 $n \leq 25$, $Q \leq 100$ 。

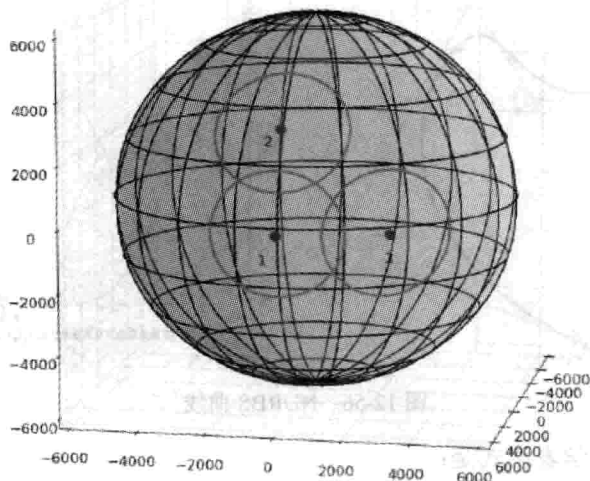


图 12-55 “最短飞行路径”问题示意图

【分析】

虽然这个题一看就是最短路径问题，但是构图才是本题的难点。假设已经成功构图，剩下的问题就是：有 n' 个点的图 G ，其中有 n 个点是特殊点（机场）。给定起点 s 和终点 t ，找一条最短路，使得路径上任意两个相邻特殊点的距离不超过 c 。首先以特殊点出发做单源最短路，求出每两个特殊点之间的最短路，然后构造一个新图 G' ，结点是特殊点，边 $u-v$ 的长为 G' 上 $u-v$ 的最短路。最短路大于 c 时不加这条边。

图 G 的结点是所有机场和每个机场的“保护圈”的交点。一共有 n 个保护圈，交点数不超过 600 个 ($2C(n, 2) \leq 600$)。对于任意两个点，当且仅当二者可以“直达”时连一条边。“可以直达”意味着它们之间的大圆弧是安全的，即这个大圆弧完全位于所有保护圈的“并”的内部。注意这个大圆弧的不同部分可能会在不同机场的保护圈内，所以不能简单地取弧的中点后依次判断每个保护圈。

判断一条大圆线^① a 是否安全的正确方法是：对于每个保护圈 s ，求出 a 被 s 保护的范，然后把所有范围求并，看看是否是完全覆盖 a 。保护圈交点的个数是 $O(n^2)$ ，因此“需要判断是否安全”的大圆弧个数是 $O(n^4)$ 。对于 $O(n)$ 个保护圈，求交点和区间并需要 $O(n \log n)$ 时间，因此总时间复杂度为 $O(n^5 \log n)$ 。

12.2.5 非完美算法

例题 12-26 可爱的魔法曲线 (Lovely M[a]gical Curves, Rujia Liu's Present 6, UVa12565)

NURBS 曲线是一种可爱而又“有魔法”的曲线。它的样子多变，非常灵活，如图 12-56

^① 大圆 (Great Circle) 是球面上半径等于球体半径的圆弧。连接两点的最短“球面线段”等于经过两点的大圆上的劣弧。

所示。

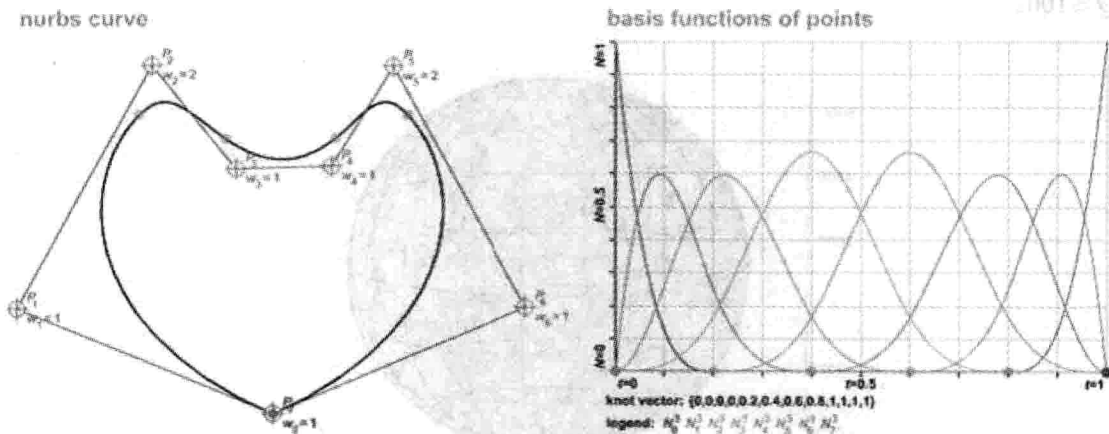


图 12-56 NURBS 曲线

NURBS 曲线的数学表达式是：

$$C(u) = \frac{\sum_{i=1}^n w_i N_{i,k}(u) P_i}{\sum_{i=1}^n w_i N_{i,k}(u)}$$

其中， u 是参数， n 是控制点个数， k 是曲线的度数， P_i 和 w_i 是第 i 个控制点的位置和权重。在上式中（计算过程中遇到的 0/0 按 0 算）：

$$N_{i,k}(u) = \frac{u - t_i}{t_{i+k} - t_i} N_{i,k-1}(u) + \frac{t_{i+k+1} - u}{t_{i+k+1} - t_{i+1}} N_{i+1,k-1}(u)$$

$$N_{i,0}(u) = \begin{cases} 1, & \text{if } t_i \leq u < t_{i+1} \\ 0, & \text{else} \end{cases}$$

NURBS 曲线的参数有严格的限制：

- 度数是正整数。
- 控制结点至少有 $k+1$ 个，和曲线形状有直接关系。
- Knot 向量为 $[t_1, t_2, \dots, t_m]$ ，其中 $m=n+k+1$ 。相邻 knot 值满足 $t_i \leq t_{i+1}$ ，定义了曲线中参数 $[t_i, t_{i+1})$ 的部分。整个 NURBS 曲线的定义域是 $[t_1, t_m)$ 。

要求求出两条 NURBS 曲线的所有交点。 $n \leq 20$ ，度数为 1, 2, 3 或者 5，控制点坐标范围是 $[0, 10]$ ，权值范围 $(0, 10]$ ，Knot 向量的第一个数保证为 0，最后一个数保证为 1。

输入保证 NURBS 曲线不病态，且没有特别接近的交点，输出保留 3 位小数。

【分析】

NURBS 曲线和曲面是工业中常用的建模工具，也是工作中实际会用到的。NURBS 曲线的定义看起来比较吓人，但仔细观察后可以发现，它实际上就是一个分段多项式曲线，可以用数学归纳法证明。 $N_{i,0}(u)$ 是分段 0 次曲线（当 u 在 t_i 和 t_{i+1} 之间时为 1，其他时候为 0），而 $N_{i,k}(u)$ 由两部分相加得到。注意， $N_{i,k-1}(u)$ 和 $N_{i+1,k-1}(u)$ 的第二个下标都是 $i-1$ ，而且系数都

是 u 的一次函数, 因此 $N_{i,k}(u)$ 比 $N_{i,k-1}(u)$ 的次数要大 1。

看清楚定义之后, 至少可以做一件事: 对于一个给定的参数 u , 计算曲线中参数 u 所对应的点, 即 $C(u)$ 。于是, 第一个算法诞生了: 对一条 NURBS 曲线, 有一个很大的正整数 p , 取步长 $s=1/p$, 然后对于参数 $i=0,1,2,\dots,n-1$ 各求出一个点 $P_i=C(is)$ (想一想, 为什么不计算 $P_n=C(1)$)。只要 p 足够大, 折线 $P_0-P_1-\dots-P_{p-1}$ 可以很好地逼近一条 NURBS 曲线。这样, 用两条折线分别逼近两条 NURBS 曲线, 然后求出两条折线的交点即可。如何求两条折线的交点? 因为交点很少, 采取《训练指南》中介绍的扫描法, 可以在 $O(p\log p)$ 时间内完成这个任务。

这个方法看上去非常不优美, 但是它可以解决问题。学习算法的目的不正是要解决问题吗? 在更好的算法被找到之前, 应该尽可能地解决问题, 不要轻易放弃。

上述方法只是一个基本梗概, 有许多细节可以优化。例如, 可以用二分法来“自适应”地构造折线, 而不是像刚才那样均分参数空间。还可以不用扫描法, 而是把 x 轴划分成一些相互重叠的小窄条, 在每个窄条里寻找交点^①。只要仔细选取上述方法的参数, 就能更快、更准地找出所有交点, 并且不会遗漏。

例题 12-27 奇怪的歌剧院 (A Strange Opera House, UVa11188)

昨天晚上, 我做了一个奇怪的梦, 梦到我站在一个多边形的歌剧院舞台上演唱。我的声音最多能被歌剧院的墙壁反射 k 次, 如图 12-57 中的 4 幅图描绘了声音的反射方式, 分别为歌剧院轮廓、声音直射的可达区域、声音反射一次的可达区域、声音反射两次的可达区域。

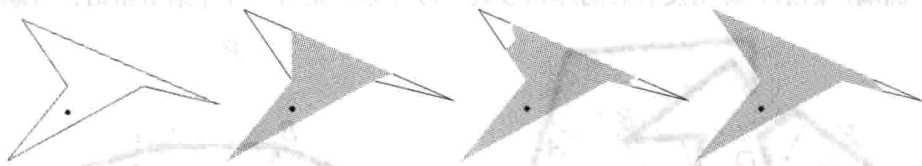


图 12-57 声音的反射方式

观众都坐在墙边。你能帮我计算一下, 有多少观众能听到我的歌声吗?

每组数据第一行为 4 个整数 n, k, x, y ($3 \leq n \leq 50$, $0 \leq k \leq 5$), 其中, n 为歌剧院多边形的顶点数, k 为最大反射次数, (x,y) 为我唱歌的位置 (保证严格在多边形的内部, 不在墙上)。以下 n 行每行为歌剧院的一个顶点坐标。顶点按照顺时针或逆时针排列。所有坐标均为绝对值不超过 1000 的整数。对于每组数据, 输出能听到我的声音的观众所对应的墙的总长度, 保留两位小数。

【分析】

本题只需要按照题目意思反射声音, 然后求出声音到达的墙的总长度即可。但这个概念上简单的过程却并不容易转化成程序。因为歌剧院是不规则多边形, 声波在传播过程中可能经过多次反射, 而且不同的声波的“反射序列” (即每次发生反射时由墙编号组成的序列) 可能完全不同。幸运的是, 这些声波依然是可以“离散化”的, 即按照角度划分成若干区间, 使得每个区间中声波的反射序列相同, 如图 12-58 所示。

^① 比赛中唯一通过此题的 Anton Lunyov 就是采用的这种方法。

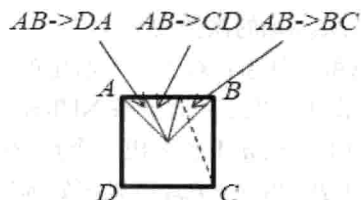


图 12-58 声波的反射序列

这样的“离散化”方案虽然概念正确，但是很难像其他题目那样通过一次预处理完成，因为要事先考虑所有可能的反射序列（多达 50^5 种）。一种折中的方案是用深度优先搜索的方式，递归地把声波角度逐步细分。

如图 12-59 (a) 所示，从 P 点出发，角度范围为 A 到 E 的声波被分成了 4 部分： A 到 B ， B 到 C ， C 到 D ， D 到 E 。接下来递归求解即可。为了递归求解，需要把子问题设计成和原问题相同的形式，即子问题也应有一个“音源”。

如图 12-59 (b) 所示，从 P 发出的声音，初始范围是向量 v_1 和 v_2 之间，其中向量 PA 和 PB 中间的部分反射出来的区域等价于 P 关于 AB 的对称点 P' 直射 A 和 B 点，得到的区域中在有向线段 AB 左侧的部分（这句话非常绕，请多读几遍）。这样，已经可以设计出递归过程了。参数有 5 个：已经反射的次数 f 、等价音源位置 P ，上次反射墙的有向线段 AB 和初始范围向量 v_1 和 v_2 。在递归过程中，首先把角度区间分成若干小区间，使得每个区间直射的是同一面墙，然后计算出发射后的递归参数并进行递归调用。程序细节留给读者编写。

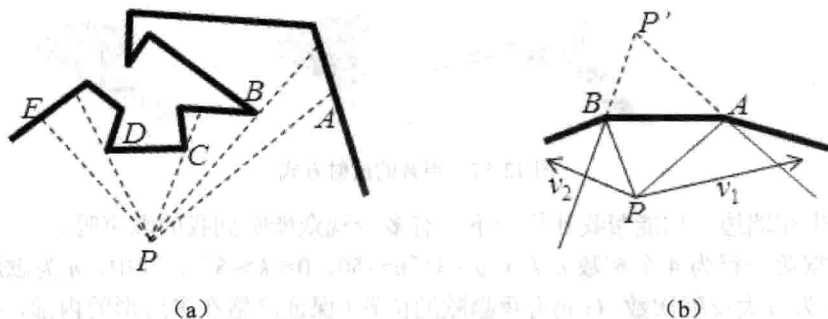


图 12-59 将声波角度逐步细分

本题还有一个姐妹篇——奇怪的歌剧院 II^①，其中把“长度”改成了“面积”，即要求计算能听到歌手声音的区域面积。有兴趣的读者可以试一试。

例题 12-28 最小包围长方体 (Smallest Enclosing Box, Rujia Liu's Present 4, UVa12308)

给定三维空间中的 n ($n \leq 10$) 个点，求一个能包含所有点的体积最小的长方体。这个长方体的各个面不一定要平行于坐标平面。只需输出最小长方体的体积。

【分析】

在《训练指南》中用旋转卡壳的方法计算了 n 个点的最小包围矩形，时间复杂度为 $O(n \log n)$ 。该方法基于这样一个定理：一定存在一个最小包围矩形（不管是面积最小还是周

^① A Strange Opera House II, Rujia Liu's Present 4, UVa12309

长最小)，贴着凸包的一条边。

对于最小包围长方体，是否有这样的结论呢：一定存在一个最小包围长方体，贴着凸包的一个面？如果这个结论成立，问题就简单多了。首先计算三维凸包，然后枚举凸包上的一个面，再整体旋转所有点，使得这个面和 $z=0$ 平面平行。这样，就可以忽略所有点的 z 坐标，求出面积最小的包围矩形 R ，则所求长方体的底就是 R ，高就是旋转之后所有点的 z 坐标最大值与最小值之差。因为 n 的范围很小，既使用最慢的三维凸包和最小包围矩形算法，也不会超时。

很可惜，上述结论是错的，即最小包围长方体不一定会贴住凸包上的一个面。如图 12-60 所示，正四面体（它的凸包是自身）就是一个反例：最小包围长方体的每个面都贴住了一条边，但是没有贴住任何一个面。

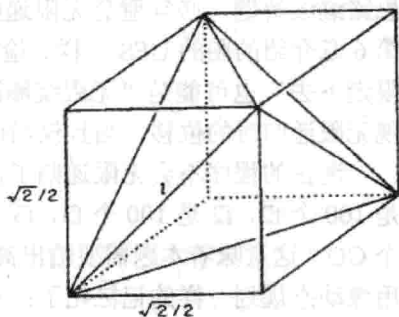


图 12-60 正四面体

事实上，已知最强的结论是：最小包围长方体中至少有两个相邻面均贴住凸包的某条边。Joseph O'Rourke 在论文《Finding Minimal Enclosing Boxes》中基于这个结论设计了一个三维旋转卡壳算法，成功地在多项式时间内解决了最小包围长方体问题，但算法很抽象、复杂，难以用到算法竞赛中。

前面曾经多次强调过，算法竞赛的目的是要解决问题。如果“正解”过于复杂，难以驾驭，可以寻找非完美解决方案。刚才的算法其实只有第一步错了，那么只要用其他办法找到最小包围长方体的一个面，还是可以用旋转、降维的方法进行求解。一个相对容易实现的方法是使用随机调整：先随机生成大量的平面，求出对应的解，然后选一些比较优秀的解进行“微调”——稍微旋转一下，如果旋转后的解更优，就更新答案。这样的随机调整方法有很多不同的实现方法，常用的一种是模拟退火方法，有兴趣的读者可以查阅相关资料。

12.2.6 杂题选讲

例题 12-29 旅行 (Journey, ACM/ICPC NEERC 2011, UVa1680)

有 n ($n \leq 100$) 个绘图函数，包含 GO（前走一步）、LEFT（左转）、RIGHT（右转）、Fk（递归调用第 k 个函数然后继续执行本函数）4 种指令。

例如程序：

```
f1: GO F2 GO F2 GO F2
f2: F3 F3 F3 F3
f3: GO LEFT
```

会画出如图 12-61 所示的图形。

有时，函数会无限执行下去，如 GO F1。

每个函数最多包含 100 条指令。从 $(0,0)$ 点开始
执行 f1，求画图过程中距离 $(0,0)$ 点最大的曼哈顿距

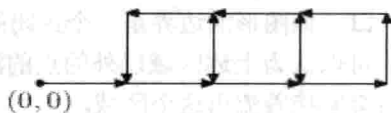


图 12-61 程序绘制的图形

离 (即 $|x|+|y|$)。如果无限大, 则输出 Infinity。

【分析】

既然题目是递归, 那么第一反应就是直接写个递归函数 $\text{simulate}(x, y, i, d)$, 表示目前在 (x, y) , 面朝方向 d , 执行函数 f_i 。在执行函数时不断更新 $|x|+|y|$ 的最大值。

可惜这样做是不行的, 因为题面已经给出了一个无限递归的例子。所以要想沿着这个思路继续解题, 必须避免无限递归。如何避免呢? 最直接的方法就是检测无限递归, 就像第 6 章介绍的图的 DFS 一样。检测到以后怎么办呢? 直接输出 Infinity? 这样可不行。“无限走下去”也可能是“无限绕圈圈”, 并不代表会离原点无限远。所以还应该记录一下出现无限递归时的位移, 当且仅当位移不是 $(0, 0)$ 时, 输出 Infinity。

现在的程序不会无限递归了, 可惜还是会超时, 因为走的步数可能会非常多。例如 f_1 是 100 个 f_2 , f_2 是 100 个 f_3 , f_3 是 100 个 f_4 , ..., f_{100} 是 100 个 GO, 则一共会执行 100^{100} 个 GO (这意味着本题需要输出高精度整数)。怎么办呢? 既然已排除了无限递归, 就可以用像动态规划一样的记忆化了: 对于 (i, d) , 记录面朝方向为 d , 执行完 f_i 之后的方向、总位移 (dx, dy) 和路径上的 $\max\{|x|+|y|\}$, 然后尝试递推。

记忆化时之所以不记录 (x, y) , 是因为它们可能会很大, 而且不同的 (x, y) , 当 i 相同时, 执行 f_i 的路线“形状”都是一样的, 因此位移也一样。可新的问题又出现了: $\max\{|x|+|y|\}$ 无法递推。具体来说, 就是设位移为 (x_0, y_0) 时, 无法根据 $\max\{|x|+|y|\}$ 计算出 $\max\{|x+x_0|, |y+y_0|\}$ 。

解决方法也非常巧妙。分别记录 $x+y, -x+y, -x-y, x-y$ 这 4 个表达式的最大值。因为没有绝对值符号, 这 4 个值是可以递推的; 当计算最终答案时, 这 4 个值的最大值就是 $\max\{|x|+|y|\}$ (想一想, 为什么)。

例题 12-30 下雨 (Rain, ACM/ICPC World Finals 2010, UVa1097)

有一个由许多不同形状的三角形沿边相互拼接而成的立体地形图, 其中三角形的每条边要么是地形图的边界, 要么与另外一个三角形的某条边完全重合。此时在地形图的上空开始下雨, 雨水会被困在地形图中而形成湖。要求编写一个程序来确定所有的湖, 以及每个湖水位的海拔高度。假设雨非常大, 所有湖的水位都到达了最高点。

对于一个湖, 一艘大小可以任意小但不为 0 的船可以在湖面上的任意两点间航行。如果两个湖在相接位置的水位深度均为 0, 则它们被认为是两个不同的湖。

输入第一行包含两个数 p 和 q ($p \geq 3, q \geq 3$), 分别表示地形图中点和边的个数。之后的 p 行描述每个点, 每行首先是点的名字, 接着是 3 个整数 x, y, h , 表示这个点的三维坐标, 其中 x, y ($-10000 \leq x, y \leq 10000$) 为点在地平面上的坐标, h ($0 \leq h \leq 8848$) 为点的海拔高度。接下来的 q 行描述每条边, 每行包含两个点的名字, 表示一条边的两个端点。

地形图在 xy 平面上的投影满足下列条件:

- ☐ 任意两条边只可能在端点处相交。
- ☐ 该图形是一个有许多三角形组成的连通区域。
- ☐ 该图形的边界是一个封闭的多边形, 内部没有空洞。

可以认为上述区域以外的点的海拔高度低于区域内任意一点的海拔高度, 水在流到边界后会紧接着流出这个区域。

对于每组输入, 在第一行输出数据的编号, 接下来以递增的顺序在每行输出一个湖的

海拔高度；如果没有湖，则输出一个 0。

【分析】

首先建一个图，结点是所有区域（即三角形和“外界”无限大区域）。当且仅当两个区域 u 和 v 有公共边时，在图上连一条边，权值为 u 和 v 的两个公共顶点的较低高度，表示只要水位高于这个高度，水就可以从 u 流到 v ，或者从 v 流到 u 。

下面这一步需要点创造性思维：考虑水从某一个区域流到“外界”的路径。这条路径上的最大权重对应着一个“最小高度”，当水位达到这个高度时，水就可以顺着这条路径流到外面。但是水可以有多条通往外界的路径，只要水位大于任何一条路径的最小高度，水就可以顺着这条路径流出去。这正是一个最短路问题吗，只不过路径的“长度”是最大边权而非边权之和而已。第 11 章中已经讨论过这样的“变形最短路”问题。

用 Dijkstra 算法求出以外界为起点的单源最短路（因为边都是无向的，以外界为终点相当于以外界为起点）之后，对每个区域 i 都求出了一个 $d[i]$ ，即“能流到外界的最小水位”，只要 $d[i]$ 大于区域 i 的 3 个顶点的最小高度，则说明区域 i 是有积水的，并且水位就是 $d[i]$ 。求出了水位，用 DFS 或者 BFS 把连通的积水区域合并起来成为“湖”即可。

例题 12-31 字典 (Dictionary, ACM/ICPC NEERC 2013, UVa1681)

输入 n ($1 \leq n \leq 50$) 个不同的单词（每个单词的长度为 1~10），设计一个结点数最少的树状字典，使得每个单词 w 都可以找到一条从上到下（即远离根结点）的路径，使得路径上出现的字母按顺序连接起来后可以得到 w 。如图 12-62 所示，7 到 5 是 north，16 到 12 是 eastern，29 到 2 是 european，3 到 25 是 regional，1 到 31 是 contest。

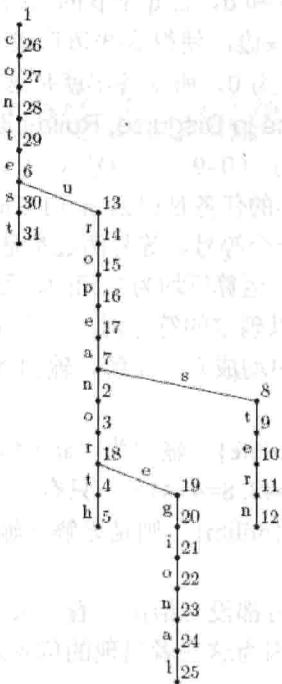


图 12-62 “字典”问题示意图

【分析】

首先把题目的要求放宽一点：必须从根开始走，而不是从任意结点开始走。这样，只需要构造这些单词的 Trie 即可，如图 12-63 (a) 所示。

这个 Trie 也可以理解成一个状态图，每个结点代表“当前得到的字符串前缀”，则本题中“从任意结点出发”的条件只需要加一些虚线边即可，如图 12-63 (b) 所示。例如，加上了 $abc \rightarrow c$ 的虚线边之后，实际上可以从根走到 abc ，然后走虚线边“扔掉前两个字符”得到 c ，这和从根直接走到 c 是完全等价的。更妙的是，从 abc 到 c 这条“边”实际上并不在最终的树状字典中，所以用它来代替从根到 c 的这一条边，能让答案更优。

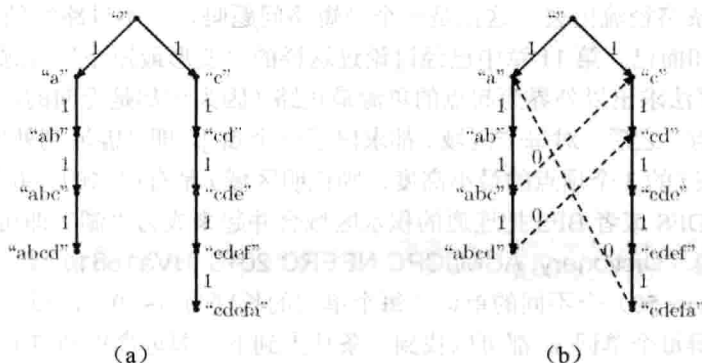


图 12-63 构造单词的 Trie

一般地，对于任意两个前缀 p 和 q ，若 q 是 p 的后缀，则连一条从 p 到 q 的虚线边。在这个图中，我们的目标是找到一些边，使得这些边形成“树状字典”，并且包含的实线边最少。设实线边权为 1，虚线边权为 0，所求答案就是这个图的最小树形图。

例题 12-32 算符破译 (Equations in Disguise, Rujia Liu's Present 1, UVa11199^①)

已知字母 $a, b, c, d, \dots, m$ 和数字 $(0 \sim 9)$ 、加号 $(+)$ 、乘号 $(*)$ 和等号 $(=)$ 之间有一个一一对应关系（一一映射）。你的任务是根据 n ($1 \leq n \leq 20$) 个等式，尽可能地推导出这些对应关系。每个等式恰好包含一个等号，等号两边都是中缀表达式，数字都是十进制的，不含前导零（但整数 0 是允许的）。运算符均为二元的，乘法的优先级比加法高（没有括号）。

对于每组数据，输出所有可以确定的符号对（一个字母和它代表的数字/运算符）。换句话说，这些符号对应在所有解中均成立。无解，输出 No；如果有解，但没有可以确定的符号对，则输出 Oops。

例如，有两个等式 $\{abcdec, cdefe\}$ ，输出为 “ $a6 \ b* \ d=f+$ ”（所有可能的解为 $\{6*2=12, 2=1+1\}, \{6*4=24, 4=2+2\}, \{6*8=48, 8=4+4\}$ ）。只有一个等式 $\{abcde\}$ ，则什么也确定不了（输出 Oops），而只有一个等式 $\{milim\}$ ，则是无解（输出 No）。

【分析】

本题的条件太苛刻，连运算符都没有给出，看上去非搜索莫属了。不难发现，应当先搜索等号、加号和乘号的位置，因为这三者出现的位置最苛刻（等号在每个等式中必须恰

^① 题目来源：NOI2000，命题人：李申杰。UVa 中的数据经过加强，难度大大高于 NOI 中的测试数据。

好出现一次，并且这三者中的任意两个都不能连续出现，也不能在等式的首尾位置）。例如，若有一个等式 $abcab$ ，则 c 肯定是等号，因为只有 c 恰好出现一次。枚举完等号以后还有一个小优化：如果某些等式在等号左右两边的字符串完全相等，则不管怎么搜，这个等式都会成立，因此只需要标记出来，今后在搜索时就可以避开无谓的判断了。

接下来搜索各个数字。 $a+b=c$ 这样的等式只需搜索 a 和 b ，则 c 就能直接计算出，所以需要重新安排各个数字的搜索顺序，使得更多的数字能够尽快直接计算出。例如， $ab+cd=ef$ 的一个较好的搜索顺序是： b, d, f, a, c, e 。其中搜索完 b, d 之后可以直接计算出 f （注意此时还要检查其他等式是否存在矛盾），而搜索完 a, c 后可以直接计算出 e 。

$abc=d+e+f$ 是不可能成立的，因为 3 个一位数加起来不可能是 3 位数。一般地，可以求出每个数的最小值和最大值，进而计算出等式两边的取值范围。例如， abc 的取值范围是 100~999（虽然不如 123~987 准确，但比较容易求）， $d+e+f$ 的取值范围是 0~27，因为 $27 < 100$ ，所以无解。这个方法有一个软肋：0 乘以任何数都等于 0，所以在 $a*b=a*cdefg$ 这样的等式里，这个方法完全不奏效。幸运的是，有一个办法可以减少这种情况的发生：先搜索 0。等 0 确定下来以后，上下界估计就会准确一些。

看上去很吸引人吧？这个剪枝的效果很不错（即可以剪掉大量枝叶），但是效率却不佳。也就是说，有可能花费大量的运行时间在“判断是否满足剪枝条件上，这就舍本逐末了。一般来说，可以尝试以下方法来调整这种“低效剪枝”：牺牲效果（即少剪一点）而提高效率，或者只在搜索的前几层才检查剪枝条件，因此那时的结点还不多，效率不会太受影响，而剪枝成功后的好处更大。

还有一个剪枝更有意思：因为并不是要找出所有解，所以如果已经 Oops 了（即有解，但所有字母都是多解），直接终止整个搜索过程即可。一般地，设 $ans(c)$ 表示“当前最终答案”中 c 的值（可能是“？”）， $val(c)$ 表示“当前解”中 c 映射到的字符（必须是 0~9 或者加号、乘号或者等号），则还没有搜索的所有字符的 ans 都是“？”，已经搜索的字符 c 满足：要么 $ans(c)=?$ ，要么 $ans(c)=val(c)$ ，即继续搜索下去，不管 val 能不能变成一个合法解，都不会改变“最终答案”。所以应该终止当前解的搜索。注意，初始时 ans 为空，此时无论如何都要先搜出一个解。

刚才的描述比较抽象，下面举一个例子。假设目前已经得到了两个解： $a=4, b=6, c=3, d=1$ ； $a=8, b=6, c=1, d=3$ ，因此 ans 是 $a=?, b=6, c=?, d=?$ 。再假设现在已经搜了 $a=2, b=6$ ，但 c 和 d 还没搜。在这种情况下不管有没有解，有何种解，都改变不了 ans 。

有了这些优化，最终的程序速度会非常快。不过本题还有一个不起眼的“陷阱”：在输入中没有出现的字符并不一定是不确定的——因为是一一映射，如果已经确定了 12 个字母，剩下的那一个也就确定了。

例题 12-33 独占访问 (Exclusive Access, NEERC 2008, UVa1682)

多线程编程中的一个重要问题就是确保共享资源的独占访问。需要独占访问的资源称为临界区 (CS)，确保独占访问的算法称为互斥协议。

在本题中，假设每个程序恰好有两个线程，每个线程都是一个无限循环，重复进行以下工作：执行其他指令（与临界区无关的代码，称为 NCS），调用 `enterCS`，执行 CS（即临界区代码），调用 `exitCS`，然后继续循环。NCS 和 CS 内的代码和协议完全无关。

在本题中，用共享的单比特变量（即每个变量只能储存 0 或者 1）来实现互斥协议（即上述的 `enterCS` 和 `exitCS`）。所有变量初始化为 0，且读写任意一个变量只需要一条语句。两个线程可以有一个局部指令计数器 `IP` 指向下一条需要执行的指令。初始时，两个线程的 `IP` 都指向第一条指令。程序执行的每一步，计算机随机选择一个线程，执行它的 `IP` 所指向的指令，然后修改该线程的 `IP`。为了分析互斥协议，定义“合法执行过程”如下：两个线程都执行了无限多条指令；或者其中一个线程执行了无限多指令，另一个线程执行了有限多条指令以后终止，且 `IP` 在 `NCS` 中。

表 12-4 中展示了 3 个互斥协议的伪代码。两个线程的 `id` 分别为 0 和 1，变量 `want[0]`、`want[1]` 和 `turn` 为共享单比特变量。以“+”开头的代码实现了 `enterCS`，而以“-”开头的代码实现了 `exitCS`。`NCS()` 和 `CS()` 表示执行 `NCS` 代码和 `CS` 代码，这些代码的具体内容和本题无关（假设它们不会修改共享变量）。

表 12-4 3 个互斥协议的伪代码

算法 1	算法 2	算法 3
<pre> loop forever NCS() + loop while + (turn == 1 - id) CS() - turn <- (1 - id) end loop </pre>	<pre> loop forever NCS() + want[id] <- 1 + loop while + (want[1 - id] == 1) CS() - want[id] <- 0 end loop </pre>	<pre> loop forever NCS() + want[id] <- 1 + turn <- (1 - id) + loop while + (want[1 - id] == 1 and + turn == 1 - id) CS() - want[id] <- 0 end loop </pre>

本题的任务是判断一个给定算法是否满足以下 3 个条件。

- ☐ 互斥性：在任意合法执行过程中，两个线程的 `IP` 不可能同时位于 `CS`。
- ☐ 无死锁：在任意合法执行过程中，`CS` 都执行了无限多次。
- ☐ 无饥饿：在任意合法执行过程中，执行了无限多条指令的线程执行了无限多次 `CS`。

互斥性很容易满足：一个什么都不干的死循环就符合条件。上述 3 个算法均满足互斥性，但前两个算法不满足“无死锁”，而第 3 个算法（由 Gary Peterson 发明）满足所有 3 个条件。

输入包含多组数据。每组数据第一行为两个整数 m_1, m_2 ($2 \leq m_i \leq 9$)，即线程 1 和线程 2 的代码行数。接下来的 m_1 行是线程 1 的代码，再接下来的 m_2 行是线程 2 的代码。每个线程的代码都是一条指令占一行。每条指令的格式如下：首先是指令编号（顺序编号为 $1 \sim m_i$ ，仅是为了可读性才放在输入中），然后是指令助记符，后面跟着若干个参数。有一种特殊的参数称为 `NIP`，即下一条指令的编号（保证为 $1 \sim m_i$ 之间的整数）。一共有 3 个单比特共享变量：`A`、`B`、`C`。指令助记符有以下 4 种。

- ☐ `NCS`：非临界区代码。唯一的参数是 `NIP`。
- ☐ `CS`：临界区代码。唯一的参数是 `NIP`。

□ SET: 写入共享变量。包含 3 个参数 v, x, g 。 v 是写入的变量 (A, B 或 C), x 是写入的值 (0 或 1), g 是 NIP。

□ TEST: 读取共享变量并判断它的值。包含 3 个参数 v, g_0, g_1 , 其中 v 是读取的变量 (A, B 或 C), g_0 是 $v=0$ 时的 NIP, g_1 是 $v=1$ 时的 NIP。

在每个线程的代码中, NCS 和 CS 恰好各出现一次。代码不一定是一个典型的无限循环, 但保证交替执行 CS 和 NCS。输入结束标志为文件结束符 (EOF)。

对于每组数据, 输出 3 个字母 Y 或者 N, 分别表示是否满足互斥性、无死锁和无饥饿条件。

【分析】

这是一道难题, 即使在 NEERC 这样高水平的区域赛中, 也只有一个队伍在比赛时通过此题。在考虑核心算法之前, 要先把程序存起来 (假设程序编号为 0 和 1)。一个合理的数据结构是保存每条指令的字母 c , var , $op1$, $op2$ 和 nip , 然后定义本题的“状态”为三元组 ($ip0, ip1, vars$), 即两个程序的“当前指令编号”以及 3 个变量的值 (最多只有 $2^3=8$ 种取值)。

接下来可以写一个 $Next(state, p)$ 函数, 即从状态 $state$ 开始让程序 p 执行一条指令以后达到的新状态, 然后从初始状态开始 BFS/DFS, 得到所有可能达到的状态, 设为 $states$ 数组。接下来的所有讨论都针对这个状态集。为了方便分析时间复杂度, 设一共有 n 个可达状态。根据上面的讨论, $n \leq 9 \times 9 \times 8 = 648$ 。

本题的 3 个定义各不相同, 下面分别验证。首先推敲一下“合法执行过程”的定义: “两个线程都执行了无限多条指令, 或者其中一个线程执行了无限多指令, 另一个线程执行了有限多条指令以后终止, 且 IP 在 NCS 中”。也就是说, 至少有一个线程会无限循环下去。对应到此处“状态”中, 这表明状态会无限转移下去。但是在无限循环过程中如果有一个程序的 IP 始终没有变化, 这个 IP 必须在 NCS 中。

exclusion 的判定。这个相对比较容易, 在计算可达状态集的同时顺便判断即可。

deadlock 的判定。回忆“无死锁”的定义: 在任意合法执行过程中, CS 都执行了无限多次。从反面看, 试着找一个执行方式, 使得从某个时刻开始 CS 再也不执行了, 这就表明出现了死锁。也就是说, 存在一个满足以下 3 个条件之一的环。

条件 1: 进入环之后, 程序 0 执行过, 但从没有到达过 CS, 而程序 1 始终停止在 NCS。

条件 2: 进入环之后, 程序 1 执行过, 但从没有到达过 CS, 而程序 0 始终停止在 NCS。

条件 3: 进入环之后, 程序 0 和程序 1 都不断执行, 且都没有到达过 CS。

starvation 的判定。和死锁类似, 饥饿的出现意味着某程序执行了无数条语句, 但只有有限多次 CS。也就是说, 存在一个环, 使得在该环中某程序曾经执行过, 但没到达过 CS。

主算法。既然死锁和饥饿都可以归结为找一个满足特定条件的环, 可以枚举环的起点 s_0 , 然后用 DFS 找环。由于判定条件比较复杂, 需要在 DFS 过程中加几个参数, 用来记录各个条件是否满足。具体来说, 可以编写递归过程 $dfs(s, m_0, m_1, c_0, c_1)$, 表示当前状态为 s , m_i 表示程序 i 有没有被执行过, c_i 表示程序 i 是否执行过 CS。当 $s=s_0$ 且 m_0 和 m_1 至少有一个为 true (说明找到圈) 时判断。

情况一: 两个程序都执行过 ($m_0=m_1=true$)。如果两个程序中至少一个没进过 CS (即 $!c_0 || !c_1$), 说明发生饥饿; 如果两个程序都没进过 CS (即 $!c_0 \& \& !c_1$), 说明发生死锁。

情况二: 存在 $0 \leq p \leq 1$ 使得程序 p 始终在 NCS (即 $m_p = \text{false}$ 且 s 状态中程序 p 在 NCS) 且程序 $1-p$ 没进过 CS ($c_{1-p} = \text{false}$), 则同时发生死锁和饥饿。

对于每个确定的起始状态 s_0 , dfs 需要 $O(n)$ 时间, 因此总时间复杂度为 $O(n^2)$ 。

例题 12-34 压缩 (Compressor, UVa11521)

你的任务是压缩一个字符串。在压缩串中, $[S]k$ 表示 S 重复 k 次, $[S]k\{S_1\}t_1\{S_2\}t_2\cdots\{S_r\}t_r$ ($1 \leq t_i < k, t_i < t_{i+1}$) 表示 S 重复 k 次, 然后在其中第 t_i 个 S 后面插入 S_i 。这里的 S 称为压缩单元。压缩是递归进行的, 因此上面的 $S, S_1, S_2, \dots$ 也可以是压缩串。你的任务是使得压缩串的长度最小。

例如, `I_am_WhatWhat_is_WhatWhat` 的最优压缩结果是 `I_am_[What]4{is}2`。注意, 上述 $k, t_1, t_2, \dots$ 的长度均算作 1, 即使它们的十进制表示中包含超过 1 个数字。一个递归压缩的例子是 `aaaabaaaaaaabaaaaaaabaaaa`, 最优结果是 `[[a]8{b}4]3`, 长度为 11。

输入包含不超过 20 组数据。每组数据包含不超过 200 个可打印字符, 但不含空白字符、括号 (小括号 `()`、方括号 `[]` 或者花括号 `{}` 都算括号) 或者数字。字母是大小写敏感的。

对于每组数据, 输出长度和压缩串。如果有多解, 任意输出一个压缩串即可。

【分析】

这是一道很难的动态规划题目, 思路不难想到, 但是细节处很容易想复杂或者写错。建议读者先自行思考一下, 写一个程序试试, 然后再阅读下面的题解。

设输入串为 A 。令 $f(x, y)$ 表示字符串 $A[x \cdots y]$ ^① 的最短压缩长度, 则有两种状态转移方式: 一是连接, 只需枚举划分点 m , 转化为 $f(x, m) + f(m+1, y)$ (如图 12-64 所示); 二是压缩, 需要枚举压缩单元的长度 L , 转移到 $f(x, x+L-1) + 3 + g(x, y, L)$, 这里的 “+3” 是方括号和数字 k , $g(x, y, L)$ 是指: 用 $A[x \cdots x+L-1]$ 作为单元来压缩 $A[x \cdots y]$ 时, 后面的 $\{S_1\}t_1 \cdots \{S_r\}t_r$ 部分的最短长度。

注意, 这个 L 必须满足 $A[x \cdots y]$ 的前 L 个字符等于后 L 个字符, 因为 $t_i < k$, 即不允许在最后面添加字符串。用 $O(n^2)$ 时间预处理出任意两个位置 i 和 j 开始的 LCP (最长公共前缀) 长度 $\text{lcp}[i][j]$ 之后, 则 L 满足条件, 当且仅当 $\text{lcp}[x][y-L+1] \geq L$ 。

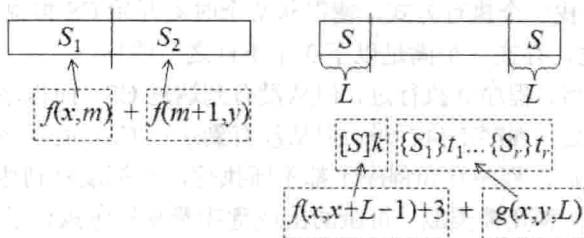


图 12-64 连接

如何求解 $g(x, y, L)$? 同样需要进行动态规划。

首先枚举压缩单元下一次出现的位置 i (需要满足 $\text{lcp}[x][i] \geq L$), 如果中间有缝隙 ($i > x+L$), 则说明有插入串 $A[x+L, i-1]$ (如图 12-65 所示), 需要递归压缩插入串 (长度为 $3 + f(x+L, i-1)$)。然后问题转化为了 $g(i, y, L)$, 即压缩 $A[i, y]$, 压缩单元为 $S[i \cdots i+T-1]$ 。

^① 习惯上用 $A[x \cdots y]$ 表示子序列 $A[x], A[x+1], \dots, A[y]$, 后同。



图 12-65 有插入串

这样，综合 f 和 g 的状态转移方程，就可以求出最优解的长度了。如何输出方案？用递归比较方便，写起来和动态规划部分类似，只是当发现当前解和最优解一样时立即递归打印。需要注意的是，在输出 f 的方案时，要先得到 g 部分的方案，同时统计单位串的重复次数，然后再输出。

算法的理论时间复杂度为 $O(n^4)$ ，但因为 L 的选取有限制，实际上效率很高。

例题 12-35 公式编辑器(Formula Editor, UVa12417)

你的任务是编写一个类似于 MathType 的公式编辑器。从技术上讲，公式就是一个表达式，它是由元素组成的序列。有 3 种元素：基本元素（算术运算符、括号、数字和字母）、矩阵和分式。

公式编辑器为每个表达式创建了一个看不见的编辑框。由于矩阵中的每个单元格都是表达式，所以每个单元格也都有一个编辑框。类似地，每个分式的分子和分母分别有一个编辑框。

在如图 12-66 所示的表达式中，有 5 个编辑框。F1 包围了整个表达式，F2 和 F3 各包围一个矩阵单元格，F4 包围了分子，而 F5 包围了分母。

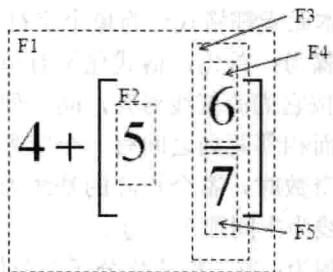


图 12-66 表达式中的编辑框

不难发现，编辑框相互嵌套。如果编辑框 A 直接包含编辑框 B ，则称 A 是 B 的父编辑框（例如，在图 12-66 中，F1 是 F2 和 F3 的父编辑框，F3 是 F4 和 F5 的父编辑框）。如果 A 和 B 拥有相同的父编辑框，则称 A 和 B 是兄弟（例如，在图 12-66 中 F4 和 F5 是兄弟，F2 和 F3 也是兄弟）。

下面介绍光标移动的实现。在任意时刻，光标总是直接包含在某个编辑框中。它可能位于该编辑框中所有元素的左边（即“框首”），也可能位于所有元素的右边（即“框尾”），还可能位于某两个相邻元素之间。如果光标在元素 X 和元素 Y 之间，并且 X 在 Y 的左边，则称光标的左相邻元素为 X ，右相邻元素为 Y 。

光标支持 6 种移动方式：Up、Down、Left、Right、Home 和 End。假定直接包含光标的编辑框为 A ，则各种移动方式的细节如下。

Home (End)：把光标移到 A 的框首（框尾）。注意，光标仍然被 A 所直接包含。

Up (Down)：如果 A 的上（下）方有一个兄弟 B，则把光标移动到 B 的框首，否则检查 A 的父编辑框。如果 A 的父编辑框有这样一个兄弟，则继续移动光标会移到该兄弟编辑框上。如果 A 的所有祖先编辑框均不含这样的兄弟，则忽略此命令。

Left (Right)：有以下 4 种情况。

- 如果光标在 A 的框首（框尾），则把它放到 A 的左（右）兄弟 B 的框尾（框首）。如果没有这样的 B，把光标放到 A 的父编辑框 C 中（如果存在），紧挨着 A 的左边（右边）。
- 如果光标的左（右）相邻元素是一个分式，把它放到分子的框尾（框首）。
- 如果光标的左（右）相邻元素是一个 n 行 m 列的矩阵，把它放到第 $\lceil n/2 \rceil$ 行第 1 列（第 m 列）的编辑框的框尾（框首）。
- 如果光标的左（右）相邻元素是一个基本元素，把它放到该元素的左（右）相邻位置。

输出格式化。本题的输出为 ASCII 格式，因此需要把每个编辑框格式化成 ASCII 字符矩形（尽管多数字符都是空格）。表达式的字符矩形由组成它的各个元素的字符矩形（称为内矩形）经过水平拼接而成。各个内矩形根据基线进行对齐，相邻两个矩形之间没有空白，而内矩形和整个矩形的边界之间也没有空白。

每个元素都可以格式化为一个字符矩形，规则如下：

- 基本元素恰好占一行，该行也是它的基线。用“-”（注意前后各有一个空格）来表示减号，而其他基本元素都格式化为单个字符。
- 矩阵元素的格式化步骤为：首先，格式化所有单元格，然后排成一个矩阵，同一行的各个 ASCII 矩形按它们的基线对齐，同一列的 ASCII 矩形水平对齐，相邻两行之间有一个空行，而相邻两列之间有一个空列；最后，在每行的前后分别加一个方括号。当行数为奇数时，整个矩阵的基线为中间那一行的基线；当行数为偶数时，整个矩阵的基线为中间那个空行。
- 分式元素的格式化步骤为：首先格式化分子和分母，然后在中间画一条水平线（由一些连续的“-”字符组成）。这也是整个分式的基线，这一行的宽度等于分子分母的较大宽度加 2（即前后各加一个字符）。分子和分母水平对齐。

前面提到的“水平对齐”是这样的：首先把水平宽度最大的矩形固定下来，然后水平移动其他矩形，使得它们的水平中心线尽量整齐。如果对齐不齐（即该矩形的宽度和最大宽度的奇偶性不同），可以往左移动 0.5 个单位的宽度，如图 12-67 所示。



图 12-67 向左移动 0.5 个单位宽度

注意有一个特例：当整个表达式为空时，ASCII 矩形是一个空行——它的宽度为 0，但高度为 1。这一点在拼接和对齐时尤为重要。

输入处理。输入已转化为了一个命令字符串序列。对于每个字符串：

- ❑ 如果它是单个字符，说明它是一个基本元素。在光标处插入此元素，然后把光标移动到它的右相邻位置。
- ❑ 如果是字符串 Matrix (Fraction)，在光标处插入一个 1 行 1 列矩阵（空分式），然后将光标右移一次。注意，光标右移之前，新的矩阵（分式）在光标的右相邻位置。
- ❑ 如果是字符串 AddRow (AddCol)，首先找到直接包含光标的矩阵，然后在最上方（最左方）添加一行（一列），并把光标移动到此行（列）中，保持列（行）不变。

如果直接包含光标的编辑框 A 并不是矩阵的单元格，需要检查 A 的父编辑框，直到找到一个矩阵。如果找不到，忽略此命令。

- ❑ 如果是字符串 Home、End、Left、Right、Up、Down 之一，按前述规则移动光标。

输入包含多组数据，每组数据以命令 Done 结束。单个数据包不超过 1000 条命令，输入总大小不超过 200KB。

【分析】

这道题目的主要难点是理清思路，建立合理的数据结构，使得编程难度、调试难度都达到一个不错的平衡点。

相关概念。题目中定义的主要概念有两个：元素和编辑框（即表达式），其中元素有 3 种：基本元素（单个字符）、分式和矩阵。这两个概念是交织在一起的，因为每个元素都有一个或多个编辑框，而编辑框就是一个或多个元素的有序序列。这里有个特别容易搞错的地方：元素的外面是没有编辑框的。例如，题目中的例子，4、“+”和矩阵外面都没有编辑框。6/7 的外面有编辑框 F3，但那是因为矩阵的每个单元格自带一个编辑框，如图 12-68 所示。

每个编辑框有一个“父元素”，而每个元素都有一个“父编辑框”，整个结构是一棵有两种结点的树。题目中的例子对应如图 12-69 所示。

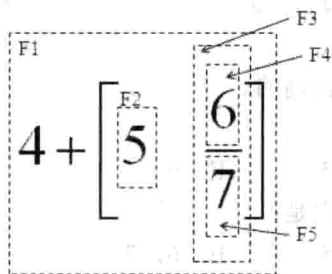


图 12-68 元素外无编辑框

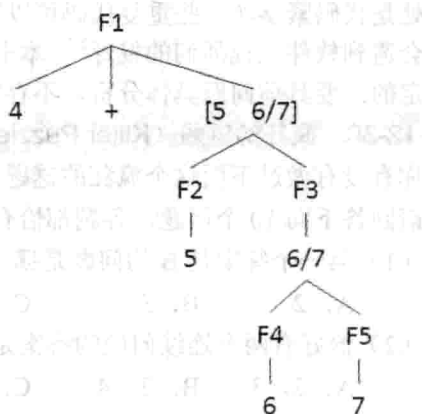


图 12-69 父元素与父编辑框

因为很多操作涉及在编辑框中寻找“上一个元素”、“下一个元素”和“首尾元素”的操作，而且还有插入元素的操作，所以编辑框可以用链表来实现。光标要么位于编辑框的尾部，要么位于某个元素 e 的前面，则光标位置实际上可以表示为 e 的指针^①。

^① 为了方便，还可以保存光标在每个级别的编辑框的元素指针。

另外，父元素相同的编辑框可以组织成十字链表（即有上下左右 4 个指针），从而支持快速的光标移动。当然，也可以写 4 个函数，动态计算每个编辑框上下左右的编辑框。这样，可得到如下的数据结构：

- ❑ 元素和编辑框都有一个父亲指针，其中元素的父亲是编辑框，编辑框的父亲是元素。
- ❑ 每个编辑框中保存“第一个子元素”和“最后一个子元素”，而每个元素中保存“下一个元素”和“上一个子元素”。
- ❑ 每个元素保存一些子编辑框，每个编辑框保存上下左右 4 个“兄弟”编辑框。这里的“一些”需要注意。基本元素只有一个框，分式也只有两个框，但是矩阵元素不仅会有多个子编辑框，而且个数还会动态改变。最容易想到的方法是直接定义一个编辑框的二维数组，但是占用空间较大。推荐的方法是只保存每行每列的首尾元素，通过十字链表访问其他元素。

格式化输出。编辑框和元素都可以进行格式化输出，也有两种常见的思路。一是递归计算出所有子结点的格式化结果，得到二维字符矩阵，然后把这此字符矩阵拼起来。这样做的好处是直观，坏处是需要大量的字符复制。第二种方式是提供两个函数，一是计算尺寸，二是以某个点为左上角把字符矩阵“画”到一个固定的字符矩阵中。这样，格式化某个结点时，先计算所有子结点的尺寸，进行排版，得到每个子结点左上角的坐标，然后让每个子结点“绘制”自己（即写到一个叫 output 的全局二维数组中）。这种方法最大的好处是避免了大量的字符复制，也是常见 GUI 软件实现布局的方法。

落实到程序上，最传统的方法是使用面向对象程序设计方法（OOP），设计两个类 Element 和 EditBox，以及 Element 的 3 个子类：Character、Fraction 和 Matrix。还有一种不很“优美”但很实用的方法：把所有类合在一起为 Object，通过一个名为 type 的字段加以区别。例如，type=0 表示编辑框，type=1、2、3 分别表示基本元素、分式和矩阵。这样做的好处是代码紧凑（一些重复代码可以写在一起）^①，坏处是代码看上去没那么好维护，而且还会遭到软件工程师们的批评^②。本书是算法书籍，无意讨论这些工程性问题，但有一点是肯定的：要具体问题具体分析，不存在适用于所有场合的“银弹”^③。

例题 12-36 疯狂的谜题（Killer Puzzle, UVa12666）

你有没有做过下面这个疯狂的谜题^④？

请回答下面 10 个问题，各题都恰有一个答案是正确的。

(1) 第一个答案是 B 的问题是哪一个？

A. 2 B. 3 C. 4 D. 5 E. 6

(2) 恰好有两个连续问题的答案是一样的，它们是：

A. 2, 3 B. 3, 4 C. 4, 5 D. 5, 6 E. 6, 7

(3) 本问题答案和哪一个问题的答案相同？

A. 1 B. 2 C. 4 D. 7 E. 6

^① 它可以把代码压缩到 5~6KB。而传统的 OOP 写法往往需要 8~10KB。

^② 这些批评也是有道理的。事实上，很多 ACM/ICPC 选手因为过于习惯编写独立、简短的代码，在工作初期会不适应大型软件的协作开发。

^③ 在软件工程领域，不同的遗留代码情况、团队情况以及软件的预计规模、需求变化情况等，都会影响到程序架构和设计决策。

^④ 相信看过《算法艺术与信息学竞赛》的读者对这个题目不陌生。

(4) 答案是 A 的问题的个数是:

A. 0 B. 1 C. 2 D. 3 E. 4

(5) 本问题答案和哪一个问题的答案相同?

A. 10 B. 9 C. 8 D. 7 E. 6

(6) 答案是 A 的问题的个数和答案是什么的问题的个数相同?

A. B B. C C. D D. E E. 以上都不是

(7) 按照字母顺序, 本问题的答案和下一个问题的答案相差几个字母?

A. 4 B. 3 C. 2 D. 1 E. 0 (注: A 和 B 相差一个字母)

(8) 答案是元音字母的问题的个数是:

A. 2 B. 3 C. 4 D. 5 E. 6 (注: A 和 E 是元音字母)

(9) 答案是辅音字母的问题的个数是:

A. 一个质数 B. 一个阶乘数 C. 一个平方数
D. 一个立方数 E. 5 的倍数

(10) 本问题的答案是:

A. A B. B C. C D. D E. E

注意:

(1) 你的答案不能自相矛盾。例如, 第一题的答案不能是 B。

(2) 你需要确保每道题的选项中只有你的答案是正确的, 其他都是错误的。例如, 若问题 (5) 的答案是 A, 那么问题 (6)、(7)、(8)、(9) 的答案都不能是 A。

(3) 你需要确保每道题目都是有效的。例如, 若问题 (2) 和问题 (3) 的答案相同, 且问题 (8) 和问题 (9) 的答案也相同, 则问题 (2) 是非法的, 因为并不是恰好有两个连续问题的答案一样。

这道题目当然可以手算, 但是作为程序员, 编程求解会更有意思。

编程求解。最容易想到的方法就是穷举法, 即考虑所有 $5^{10} = 9765625$ 种可能, 依此检查答案是否合法 (即每道题有且只有你的答案是正确的)。伪代码如下:

```
forall(answer_list):
    bad = False
    for testing_question in [1,2,3,4,5,6,7,8,9,10]:
        for testing_option in ["a","b","c","d","e"]:
            # your answer should be correct
            if testing_option == answer_list[testing_question] and
            check(testing_question, testing_option) == False:
                bad = True
            # other options must be incorrect
            if testing_option != answer_list[testing_question] and
            check(testing_question, testing_option) == True:
                bad = True
```

```
if not bad:
    print answer_list
```

在上述伪代码中，`answer_list` 是一个字母列表（下标从 1 开始），其中第 i 个字母表示第 i 个问题的答案。本题的唯一解是 `cdebeedcba`（如果每道题目的答案前加上题目编号，它是 `1c2d3e4b5e6e7d8c9b10a`）。

是不是很有趣？还有更神奇的。你可以写一个更加通用一些的程序，以求解其他类似的谜题，而不仅仅是解上面这一个谜题。不过在此之前，需要把问题描述加以形式化。

问题的形式化描述。 本题采用一种 LISP 方言来描述谜题。LISP 的语法很简单。`(f a b)` 表示用参数 a 和 b 调用函数 f ，相当于 C/C++/Java 的 $f(a, b)$ 。类似地，`(f a (g b c) d)` 相当于 C/C++/Java 中的 $f(a, g(b, c), d)$ 。下面是一道问题的例子：

```
3. (equal (answer 3) (answer (option-value)))
a. 1
b. 2
c. 4
d. 7
e. 6
```

上面的问题涉及两个重要的内置函数，如表 12-5 所示。

表 12-5 两个重要的内置函数

函 数	说 明
<code>(answer idx)</code>	返回伪代码中的 <code>answer_list[idx]</code>
<code>(option-value)</code>	返回伪代码中 <code>testing_option</code> 的计算结果（即把它看作一个表达式）

在上面的例子中，如果 `testing_option` 的计算结果是 `c`，则 `(option-value)` 返回 4（整型），因为 4 是选项 `c` 所对应的计算结果。注意，`testing_option` 的文本可以是一个复杂的表达式，参见样例输入。

上面用到的函数 `check(testing_question, testing_option)` 可以这样实现：

```
check(testing_question, testing_option):
```

1. set-up the function `(option-value)` so that it returns the evaluation result of `testing_option` of `testing_question`
2. evaluate the lisp expression of `testing_question` (e.g. the expression `(equal (answer 3) (answer (option-value)))` in the example above)
3. if an unhandled exception is raised during the evaluation, returns `False`
4. if the result of step 2 is boolean, return it; otherwise return `False`

有一个特殊的表达式叫做 `none-of-above`，其计算结果取决于其他选项的计算结果。每个问题最多只有一个 `none-of-above` 的选项，并且一定是最后一个选项。

下面是本题所用 LISP 方言的一些细节。

□ 一共有 4 种数据类型：整型、字符串、布尔型和函数。

❑ 布尔型只有两个值：`true` 和 `false`。注意，布尔值没有常量表示方法，所以无须考虑是用 Scheme 里的 `#t` 和 `#f` 还是 Common Lisp 里的 `t` 和 `nil` 来表示布尔常量。

❑ 整型都是非负整数。

❑ 字符串都用双引号包围，例如 “a string”。

❑ 所有由字母和横线组成的字符序列都是预定义函数。没有变量。

下面是预定义函数列表。所有以 “!” 开头的函数有可能抛出异常，而以 “@” 开头的函数会处理异常。和 C++/Java/Python 一样，当异常从一个函数抛出后，表达式计算的过程将会终止，除非有该函数的调用者处理异常。

基本函数如表 12-6 所示。

表 12-6 基本函数

函 数	说 明
<code>(equal a b)</code>	返回伪代码中的 <code>answer_list[idx]</code>
<code>(option-value)</code>	上面已经讨论过
<code>!(answer idx)</code>	上面已经讨论过。如果 <code>idx</code> 不是整数或不在范围 $1 \sim n$ 内（其中 n 是问题总数），则抛出异常
<code>!(answer-value idx)</code>	返回 <code>answer_list[idx]</code> 对应的表达式的值。 <code>Idx</code> 取值非法时会抛出异常

谓词是一类特殊的函数，唯一参数是个任意类型的值，返回一个布尔值，不会抛出异常，如表 12-7 所示。

表 12-7 谓词

函 数	说 明
<code>primp-p</code>	当且仅当参数是一个正素数时返回 <code>true</code>
<code>factorial-p</code>	当且仅当参数是一个阶乘数时返回 <code>true</code>
<code>square-p</code>	当且仅当参数是一个平方数时返回 <code>true</code>
<code>cubic-p</code>	当且仅当参数是一个立方数时返回 <code>true</code>
<code>vowel-p</code>	当且仅当参数是单个字符的串，并且是元音时返回 <code>true</code>
<code>consonant-p</code>	当且仅当参数是单个字符的串，并且是辅音时返回 <code>true</code>

查询和统计函数如表 12-8 所示。

表 12-8 查询和统计函数

函 数	说 明
<code>!@(first-question pred)</code>	返回满足谓词 <code>pred</code> 的第一个问题编号 $1 \sim n$ 。如果不存在，则抛出异常
<code>!@(last-question pred)</code>	返回满足谓词 <code>pred</code> 的最后一个问题编号 $1 \sim n$ 。如果不存在，则抛出异常
<code>!@(only-question pred)</code>	返回满足谓词 <code>pred</code> 的唯一问题编号 $1 \sim n$ 。如果不存在或者不唯一，则抛出异常
<code>@(count-question pred)</code>	返回满足谓词 <code>pred</code> 的问题个数
<code>!(diff-answer idx1 idx2)</code>	返回问题 <code>idx1</code> 和 <code>idx2</code> 的答案之差（例如， <code>a</code> 和 <code>b</code> 相差 1）。返回值总是 $0 \sim m$ 的整数。如果 <code>idx1</code> 或 <code>idx2</code> 非法，则抛出异常

注意：表 12-8 中的前 4 个函数（即有 “@” 标记的函数）可以处理异常，即如果在计算 `pred`

的过程中抛出了异常,这4个函数不会把异常传递给它的调用者,而是当作 pred 返回了 false。例如,如果 answer_list 是 abc,则表达式(count-question (make-answer-diff-next-equal 0))返回 0,而不会抛出异常,尽管计算((make-answer-diff-next-equal 0) 3)时会抛出异常。注意,所有其他函数都不会处理异常,例如,若一共只有 3 个问题,则(factorial-p (answer-value 5))会抛出异常,而不是返回 false。

谓词生成器如表 12-9 所示。

表 12-9 谓词生成器

函 数	说 明
!(make-answer-diff-next-equal num)	返回一个谓词(p idx)。该谓词先计算(diff-answer idx idx+1),当计算结果等于 num 时返回 true。当 num 不是整数时抛出异常
(make-answer-equal a)	返回一个谓词(p idx)。该谓词先计算(answer idx)。当计算结果等于 a 时返回 true
(make-answer-is pred)	返回一个谓词(p idx)。该谓词先计算(answer idx)。当计算结果满足谓词 pred 时返回 true
(make-answer-value-equal a)	和上面类似。计算的是(answer-value idx)
(make-answer-value-is pred)	和上面类似。计算的是(answer-value idx)
!(make-is-multiple num)	返回谓词(p i)。该谓词返回 true 当且仅当 i 是整数且是 num 的倍数。当 num 不是整数时抛出异常
!(make-equal val)	返回谓词(p v)。该谓词返回 true 当且仅当(equal v val)为真。当 val 既不是整数也不是字符串时抛出异常
(make-not pred)	返回谓词(p v)。当且仅当(pred v)为 false 时该谓词返回 true
(make-and pred1 pred2)	返回谓词(p v)。当且仅当(pred1 v)和(pred2v)均为 true 时返回 true。注意, pred1 和 pred2 都要测试,不能进行短路操作
(make-or pred1 pred2)	返回谓词(p v)。当且仅当(pred1 v)和(pred2v)至少有一个为 true 时返回 true。注意, pred1 和 pred2 都要测试,不能进行短路操作

例如, (make-is-multiple 3)返回谓词“是 3 的倍数”,因此((make-is-multiple 3) 6)返回 true,而((make-is-multiple 3) 10)返回 false。类似地, (make-not (make-or square-p prime-p)) 返回谓词“既不是平方数也不是素数”。

输入包含不超过 50 组数据。每组数据的第一行是问题的个数 n 和选项的个数 m ($2 \leq n \leq 10$, $2 \leq m \leq 5$), 每个问题用 $m+1$ 行表示,即问题的表达式和各个选项的表达式。问题按输入顺序编号为 $1 \sim n$,选项编号为 $a \sim e$ 。选项保证是合法的表达式,并且不会调用(option-value) (否则会引起无限递归!)。每个问题后有一个空行。输入的大部分数据都是简单的。

对于每组数据,输出数据编号和所有答案,按照字典序从小到大排列,各占一行。

样例输入(节选):

```
3 3
(equal (option-value) (count-question (make-answer-equal "a")))
3
0
```

1

```

(equal (option-value) "a")
"c"
"b"
"a"

((option-value) (count-question (make-answer-equal "c")))
(make-and (make-is-multiple 2) (make-or factorial-p prime-p))
(make-not prime-p)
"none-of-above"

```

样例输出（节选）：

Case 1:

bcb

cca

【分析】

这是笔者为第9届湖南省大学生程序设计竞赛所命的一道压轴题目。本题的背景与 Lisp 相关，但为了题目的清晰简洁以及“公平”起见，有些细节与 Scheme 和 Common Lisp 不同。实际上，Common Lisp 是笔者最喜欢的语言之一^①，所以“让更多参加算法竞赛的人知道 Lisp”成为了本题的另一个目标。

本题的题干很长，不过核心内容并不多，主要是预定义函数太多。其实整个题目的意思很简单，就是用穷举法求解一个复杂的逻辑谜题。因为这个谜题的题干和选项都采用 LISP 方言来描述，而且这个方言（即预定义函数）还要足够强大到可以描述题目最初提到的那个经典谜题，所以题目的复杂程度可想而知。

主算法就是穷举所有可能的 `answer_list`，依次判断是否正确；判断 `answer_list` 是否正确的方法就是依次判断每个问题的每个选项是否满足条件——`answer_list` 中选中的选项必须正确，其他选项必须错误（还要加上对 `none-of-above` 的特判）。所以其实问题的核心在于：给定 `answer_list`，计算一个表达式。

表达式是按照字符串的格式输入的，但是为了效率，应当事先把它解析并保存在合理的数据结构中，这样才能快速求值。这个过程相当于程序设计语言的“编译”。不过这个编译的结果并不是机器指令，而是我们自己设计的内部格式，例如，一个称为 `Expression` 的类。具体来说，它有两种情况，一是常数（例如字符串、布尔值），另一个是函数调用。

每个 `Expression` 都可以计算，得到一个计算结果，因此 `Expression` 应该有一个 `eval(context)` 函数，返回一个 `Value` 类型的变量，这里的 `context` 是指“上下文”，即所有的 `question` 表达式，`option` 表达式，还有 `answer_list` 等。计算表达式所需要的所有内容都在

^① 这是一个很特别的程序设计语言，看过《黑客与画家》的读者相信对它并不陌生。这个语言有不少吸引人的地方，但它的复杂程度却是大大超过普通人的预期。对此，笔者在实际项目的开发中已略有体会。有兴趣的读者可阅读《ANSI Common Lisp》入门，然后在《On Lisp》和《Practical Common Lisp》等经典著作中找到更多信息。

context 里。

根据题意，Value 类型除了 C++ 中的 int、bool 或者字符串 char* 之外^①，还可以是函数（实际上用于“闭包”，后面还会讨论），因此需要自定义一个 Function 类。由于 Value 类主要用于承载数据，此处不再用继承的方式编写 int、bool 等子类，而是用不同的 TYPE 加以区分。例如^②：

```
struct Value {
    ValueType type; //值的类型，有 INTEGER、BOOLEAN 等
    bool boolVal;
    int intVal;
    const char * strVal;
    Function * funVal; //自定义的 Function 类
    //还有一些 GetBoolean()、GetFunction() 以及 MakeBoolean()、MakeFunction() 等函数，其作用望文知义，具体实现略
}

class Function {
public:
    virtual ~Function() {}
    virtual Value Call(const Context & c, const Value* params, int
paramsCount)=0;
};
```

对于上述代码中的技巧，特别是纯虚函数，请读者自行阅读相关资料。有了这些，就可以定义 Expression 类了。

```
class Expression {
public:
    virtual ~Expression() {}
    virtual Value Evaluate(const Context & context) = 0;
};

class LiteralExpression : public Expression {
    Value _arg;
public:
    //构造函数略
    virtual Value Evaluate(const Context &) { return _arg; }
};

class CallExpression : public Expression {
```

^①当然可以用 STL 的 string 来表示字符串。但是因为本题的字符串大都非常短，所以使用 STL 字符串带来的效率损失是比较明显的。

^② intVal、strVal 等成员可以写成联合（union）的形式以节省空间，不过和本题的核心关系不大，这里就不叙述了。

```

Expression * _functionExpression;
Expression ** _params; //也可以用 vector, 但速度稍慢, 因为最多只有两个 params
int _paramsCount;
public:
    //构造/析构造函数略
    virtual Value Evaluate(const Context & context) {
        Value fn = _functionExpression->Evaluate(context);
        if (fn.GetType() == ERROR) return Value::MakeError(); //抛出异常
        assert(fn.GetType() == FUNCTION); //必须是函数
        Value evaluatedParams[2]; //最多是二元函数
        for (int i = 0; i < _paramsCount; ++i)
            evaluatedParams[i] = _params[i]->Evaluate(context);
        return fn.GetFunction()->Call(context, evaluatedParams, _paramsCount);
    }
};

```

这里有一个地方需要特别注意: CallExpression 里的 _functionExpression 的类型是 Expression, 因此它既有可能是 LiteralExpression 又有可能是 CallExpression。例如(equal 1 1), 这里的 _functionalExpression 就是 LiteralExpression, 即 equal; 但是对于((make-equal 1) 1), _functionExpression 就是(make-equal 1), 是一个 CallExpression。

另外, 上面的代码包含了异常处理。在 Value 中增加了一种类型: ERROR。如果在计算 fn 时抛出了异常, 则整个表达式都应抛出异常。

接下来有 3 个任务: 写 Parser、编写预定义函数和编写主程序。主程序在题目中已经给出, 这里不再赘述。Parser 不难编写, 但是在处理常量表达式时要注意。根据题目, 一共只有 3 种常量表达式: 遇到数字串, 得到的 Value 是整型, 例如 10; 遇到带引号的字符序列, 得到的 Value 是字符串, 例如"none-of-above"; 遇到不带引号的字符序列, 得到的 Value 是函数, 例如 equal。换句话说, 所有预定义函数都必须是 Function 类或者它的子类, 否则无法保存到 Value 中。

因此接下来的工作重点是编写预定义函数。这个工作理论上并不困难, 但代码量大(占到总程序的一半以上), 并且容易出错。所以在编码之前, 有必要把一些细节想清楚。

之前说过, 所有预定义函数应当是 Function 类或者它的子类, 但具体来说还是有两种不同的写法。一种是写一个巨大的 PredefinedFunction 类, 保存一个 functionName, 然后在 Call 函数中根据 functionName 判断。还有一种写法是每个函数写一个单独的子类。两种写法各有利弊, 读者可以根据需要进行选用。

不管使用哪种方法, 都面临一个问题: 如何保存动态生成的函数(即闭包)。其实动态生成的函数并不是任意生成的。例如, 所有由 make-equal 生成的函数都较相似, 只是有一个参数 a 不一样。所以可以把所有“由 make-equal 生成的函数”统一处理。

如果采用方法一(即一个巨大的 PredefinedFunction 类), 可以用 functionName="generated-by-make-equal" 来表示由 make-equal 生成的函数, 另外在类中增加成员变量 a 和 functionName, 一同代表(make-equal a)的返回值。

如果采用方法二（每个函数是一个类），推荐把由 `make-equal` 生成的类写成 `MakeEqual` 函数的内部类，因为其他类都不会用到这个类。这样一来，甚至没必要给它命名。例如：

```
class MakeEqual : public Function1 {
    class _F : public Function1 { //内部类
        Value _val;
    public:
        inline _F(const Value & val) : _val(val) {}
        virtual Value Call(const Context & context, const Value & a) {
            return Equal().Call(context, a, _val);
        }
    };
    public:
        virtual Value Call(const Context &, const Value & val) {
            return Value::MakeFunction(new _F(val));
        }
};
```

上面的代码还展示了方法二的一个重要技巧：由于最多是二元函数，可以编写 `Function` 的 3 个子类：`Function0`、`Function1`、`Function2`（即有 0 个、1 个、2 个参数的类），然后让具体的函数继承这 3 个类^①。这样做可以把一些与具体函数无关的操作（例如，检查参数个数，以及是否有参数是 `ERROR` 类型）移到这 3 个类中，还可以加一些方便调试的语句，让具体函数的实现更简洁。由于本题的特殊性，还可以编写 `IntegerPredicate` 和 `StringPredicate` 两个子类，进一步地避免重复代码（主要是参数类型检查）。

至此，整个题目就分析完毕了。按照上述方法编写的代码效率很高，可以在很短的时间内通过测试数据。但优化是无止境的。如果把本题的主算法改成回溯（而非完全枚举），可以实现一个杀手级的剪枝，程序运行效率可以提高几十倍甚至上百倍。剪枝的思路如下：在 `answer_list` 没有枚举完时，虽然有些表达式无法算出结果，但有些表达式仍是能算出结果的（例如，前两题的答案确定后，`(diff-answer 1 2)` 就能算出来了）。不确定的结果可以在 `Value` 类中新增一个 `NA` 类型，然后在函数求值时判断：当函数本身和所有参数都不是 `NA` 类型时，答案也是确定性的。这个剪枝思路很直观，不过需要注意细节，有兴趣的读者可以自行尝试。

例题 12-37 太空站之谜 (Mysterious Space Station, Rujia Liu's Present 7[®], UVa12731)

3000 年的一天，人们在茫茫的宇宙中发现了一些奇怪的太空站。科学家们用高科技探测出了它们的精确位置，并绘制了地图，准备派一批机器人到那里进行深入研究。

地图是一个 $N \times M$ 的矩形网格，如图 12-70 所示每个格子要么是可以穿梭自如的真空（用白色表示），要么是无法逾越的未知物质（用阴影表示）。机器人每次可以沿着东（E）、南（S）、西（W）、北（N）中的一个方向前进到相邻格子）如果那里没有未知物质阻挡）。

^① 这个设计也许会让 `scala` 程序员会心一笑。另外，熟悉 STL 的读者也许会更倾向于复用 STL 中的 `functor`。

^② 题目来源：NOI 冬令营 2002。命题人：刘汝佳。

由于太空站内没有任何光线和其他可被机器人感知的物质，机器人只有在尝试往某一个方向行进并失败以后才能知道该方向的相邻格子无法到达，而不能事先知道某一方向上是否有障碍。

有趣的是，太空站里所有未知物质连成一片（沿东、南、西、北4个方向连通），把所有真空格围在中间，形成一个真空大厅，机器人从任何一个真空格出发都可以走到其他所有真空格中。另外，太空站内没有“狭窄的通道”，即对于每个真空格子来说，它的南北方向至少有一个相邻格子是真空，东西方向也至少有一个相邻格子是真空。为了方便，把所有的真空格按照从北到南，从西到东标号为1,2,3……。如图12-71所示就是其中一个叫FT的太空站的地图标记。

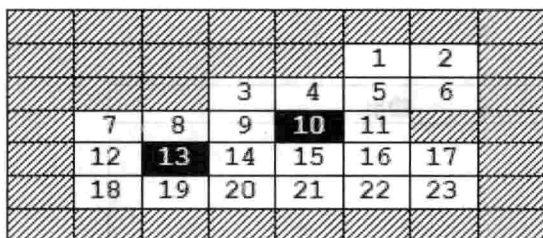


图 12-70 地图



图 12-71 FT 太空站的地图标记

机器人一号被运送到了 FT 的 12 号真空格（由于技术限制，机器人只能被运送到某个和未知物质有公共边的格子）后开始工作。机器人从起始位置出发往东走一格，再往北走一格，以为到达了 8 号格。但当它试着往北移动时，发现竟然没有被阻挡，而是成功地走到 8 号格上方那个地图上标记为未知物质的格子。这一重大发现很快传遍了所有在太空站内工作的机器人。它们一致认为地图有误，因而用集体罢工的方式向人类提出抗议。

针对这一情况，科学家们解释说：地图并没有绘制错，该现象的发生是因为太空站中存在着某种神秘的传送装置——虽然机器人一号在行走中已经被瞬间转移到其他格子中去了，但他自己却一点也感觉不到。

科学家们指出，太空站中有 K 个传送装置，每一个装置逻辑上连接着两个不同的真空格子，称为传送门。每个传送门只能属于一个传送装置，并且任意传送门周围的 8 个格子中不会有其他传送门或者未知物质。如果两个传送门属于同一个传送装置，那么当机器人沿某一个方向进入其中一个传送门，它就会被瞬间转移到另一个传送门并沿该方向再前进一步。在机器人看来，这一过程和普通的行走并没有区别，因此它们无法感知瞬间转移的进行。以 FT 为例，由于有一个传送装置连接着 10 号格和 13 号格，机器人一号的实际路线是 $12 \rightarrow 11 \rightarrow 5 \rightarrow 1$ ，根本没有到达格子 8 上面那个不能去的格子。

机器人明白了其中的奥秘以后，迫不及待地想要找出这些传送装置，但又担心自己在太空站中的工作时间会过长。经过一番慎重的考虑，科学家们决定请你编写一个智能控制程序，帮助机器人用不超过 32767 步数找到所有传送装置。

本题是一道交互式题目。对于每组数据，你的程序应当首先读入整数 N, M, K ($6 \leq N, M \leq 15, 1 \leq K \leq 5$) 的值，然后是一个 N 行 M 列的地图，其中“.”表示真空，“*”表示未知物质，“S”表示起点。起始位置保证与至少一个未知物质格有公共边，真空格保证不出

现在地图的边或角上。输入数据保证无错，行末无多余空格。

接下来，你的程序应当向标准输出打印一些移动机器人的指令，每个指令占一行，格式为 `MoveRobot D`，其中 `D` 为 4 个字符 `N`, `E`, `S`, `W` 之一。然后你的程序可以从标准输入中读到指令的执行结果，0 表示失败，1 表示成功。

算出结果之后，你的程序应当向标准输出打印恰好 K 条输出指令，每个指令占一行，格式为 `Answer pos1 pos2`，表示有一个传送装置连接真空格 `pos1` 和 `pos2`。每个传送装置应恰好输出一次，顺序任意。当所有 K 条输出完毕之后，你的程序应准备求解下一组数据测试（即再次读取 N, M, K ）。当 $N=M=K=0$ 时输入结束。

注意，向标准输出打印每一行之后必须执行 `flush` 标准输出（例如，C/C++ 可以执行函数 `fflush(stdout)`）。

如图 12-72 所示是一个交互范例。

输入	输出
7 8 1	

*S*****	

1	MoveRobot E
	MoveRobot N
1	MoveRobot N
	MoveRobot W
0	MoveRobot N
0	MoveRobot E
1	MoveRobot E
0	
0 0 0	Answer 10 13

图 12-72 交互范例

【分析】

本题是笔者第一次给正式比赛命的题目，参加现场比赛的 20 位 IOI 国家集训队员的最好成绩是解决 10 个测试点中的 2 个。

在此之前，IOI99 中出现过一道看上去类似的题目“地下城市”^①：给定一张地图，但是不知道你的当前位置。要求使用 `look` 和 `move` 指令来算出你的当前位置，其中 `look` 可以判断当前位置的某个方向是空地 `O` 还是墙 `W`，`move` 则是往某个方向移动一格。目标是 `look` 的次数尽量少。这道题目可以用筛法解决。初始时所有空地都有可能是“当前位置”，根

^① <http://olympiads.win.tue.nl/oi/oi99/contest/official/under.html>。

据 look 指令的返回值，可以排除一些可能性，当可能性只有一种时，它就是正确答案。当然，还有一些细节问题要考虑（例如，需要计算一下到哪个位置去 look 比较容易排除更多的可能性），但算法的主框架就是这样。因为最多只有 $100 \times 100 = 10000$ 个可能的位置，所以并不是很困难。

本题却是完全不同的。最多有 $11^2 = 121$ 个不与未知物质相邻的真空格，任选 5 对格子的方法有很多种（有兴趣的读者可以自己算一下），而且很难简单地通过几条指令来排除一种方案，看来需要放弃“筛法”。

怎么办呢？看来只好用逻辑思考的方法设计方案了。一开始机器人是知道自己位置的，可是走了几次以后就不知道自己在哪里了。根据题目给出的信息，移动是可逆的，即如果成功执行了移动序列 EENWN，则执行序列 SESWW 的结果一定是每步都成功，并且回到了执行 EENWN 之前的位置。有了这个结论，就不怕“走丢”了，大不了原路返回，继续下一次探索。

尽管如此，“走丢”这件事情还是应该尽量避免，因为在不知道当前位置的情况下，能获得的信息十分有限。所以机器人应当遵循以下基本原则：尽量在肯定没有传送门的格子中行走。不过，未知格子总是避不开的，因为我们必须找到传送门。如图 12-73 所示，白色格子是肯定没有传送门的，因为它们和未知物质相邻。但是灰色格子就不一定了：它们可能是传送门，也可能不是。如何判断呢？

设需要判断 A 是不是传送门。首先走到 B，然后执行移动序列 SW，则当且仅当 A 不是传送门时，移动序列 SW 可以成功，并且当前位置是 C。是否可能执行 S 时从 A 传送到另外一个位置 D，然后执行 W 时再传送回 C 呢？不可能，因为一个传送门只能属于一个传送装置，而从 D 往 W 走一步后不可能走到与 A 配对的传送门（从 D 往 N 走才能走到与 A 配对的传送门）。

这样一来，问题的关键就变成了判断当前位置是不是 C。首先，如果当前格子不“靠边”，说明它肯定不是 C，直接排除；否则可以用“单手扶墙法”来“绕圈”^①。例如，从 A 开始左手扶墙，可以得到这样一个移动序列：NESESWWN，然后回到 A。如果从 A 上面的格子出发，移动序列应当是 ESESWWNN，如图 12-74 所示。不难发现，如果把移动序列看成一个环状串，每个格子的移动序列对应的都是这个环状串的一种线性表示。换句话说，根据一个“靠墙点”的“扶墙移动序列”，就能确定这个点的具体位置。

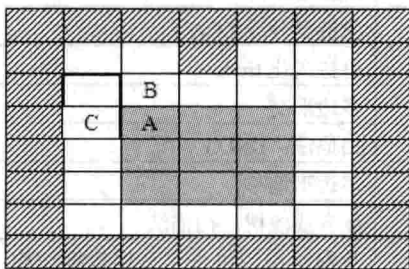


图 12-73 找到传送门

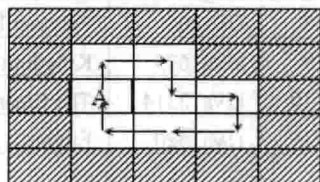


图 12-74 判断当前集团

^① http://en.wikipedia.org/wiki/Maze_solving_algorithm#Wall_follower.